

PROYECTO

Deck&Dagger: Goblin Hunt

Desarrollo de aplicaciones multiplataforma

Andrés Serrano Modrego

16/01/2026

0. Resumen Ejecutivo	3
1. Identificación del producto	4
1.1. Análisis del sector y estructura de la empresa	4
1.2. Estudio de Mercado	11
1.3. Innovación, dificultad e investigación	17
1.4. Ayudas y subvenciones	20
1.5. Especificación de requisitos	25
1.6. Casos de Uso	27
1.7. Bocetos iniciales	36
2. Diseño de la aplicación	51
2.1. Estudio de viabilidad	51
2.2. Planificación del proyecto	54
2.3. Diagrama Entidad Relación	57
2.4. Esquema Relacional	58
2.5. Diagrama de Clases	63
2.6. Diseño de interfaces	64
2.7. Plan de pruebas	75
3 Gestión de proyecto	80
3.1. Scrum	80

0. Resumen Ejecutivo

Deck & Dagger: Goblin Hunt es un videojuego del género *roguelike deck-builder* desarrollado con Godot 4.5.1, en el que el jugador construye y mejora un mazo de cartas mientras avanza por combates estratégicos por turnos contra enemigos y jefes. El juego combina toma de decisiones, gestión de recursos y progresión continua en partidas autoconclusivas.

Con su implementación se espera ofrecer al usuario final una experiencia de juego estratégica, accesible y rejugable, donde cada partida sea diferente y premie la planificación, la adaptación y la mejora progresiva del mazo.

El desarrollo de esta aplicación es importante porque permite aplicar de forma práctica y conjunta conocimientos de programación, diseño de interfaces, bases de datos y metodologías ágiles, además de responder a la creciente demanda de videojuegos independientes con mecánicas profundas y bien estructuradas.

1. Identificación del producto

1.1. Análisis del sector y estructura de la empresa

Clasificación del sector

Entretenimiento y videojuegos

El sector del entretenimiento digital engloba todas las actividades relacionadas con la creación, distribución y consumo de contenidos interactivos. Dentro de este amplio sector, los videojuegos constituyen una de las industrias más relevantes y de mayor crecimiento a nivel mundial, superando incluso en ingresos al cine y la música combinados.

El subsector del desarrollo de videojuegos se centra en el diseño, programación, arte, sonido y publicación de productos de ocio digital interactivo. Este subsector se caracteriza por una gran diversidad de modelos de negocio, escalas de producción y tecnologías empleadas.

Criterios de clasificación del sector

Dado que el sector del videojuego es muy amplio y heterogéneo, se puede clasificar a las empresas según distintos criterios estructurales y de negocio.

Tamaño de la empresa

Define la escala de producción, el número de empleados y la capacidad financiera.

Indie

Estudio independiente formado por equipos pequeños (1-10 personas). Financiación propia o crowdfunding. Ejemplos: ConcernedApe (Stardew Valley), Maddy Makes Games (Celeste), Subset GAMES (Into the Breach).

AA

Equipos medianos (20-100). Presupuesto intermedio. Suelen contar con publisher externo. Ejemplos: One More Level / 505 Games (Ghostrunner II), Spiders (GreedFall), Hazelight Studios (It takes two).

AAA

Corporaciones o filiales con cientos de empleados y gran presupuesto. Producción Internacional. Ejemplos: RockStar (Grand Theft Auto), EA (FIFA), Ubisoft (Assassin's Creed).

Plataforma y modelo de negocio

Determina el dispositivo destino y la forma en que se monetiza el juego

PC Premium

Pago único. Se distribuye en Steam, GOG, Epic. Ejemplos: Hollow Knight, Hades.

Móvil F2P

Free-to-play. Descarga gratuita, monetización mediante anuncios o microtransacciones (IAPs). Ejemplos: Clash Royale, AFK Arena.

Consola / Multiplataforma

Lanzamientos premium o híbridos. Requieren certificados y soporte técnico adicional. Ejemplos: Dead Cells, Minecraft.

GaaS

Game-as-a-Service. Juego en evolución continua, con actualizaciones periódicas. Ejemplos: Fortnite, Genshin Impact.

Alcance geográfico

Mide la escala del público objetivo y la proyección internacional.

Local

Proyecto orientado a un mercado nacional, con distribución directa o limitada.

Global

Juegos distribuidos internacionalmente, traducidos y adaptados culturalmente.

Complejidad del servicio

Se refiere a la naturaleza del producto: si es cerrado y autoconclusivo, o si se mantiene en el tiempo como servicio.

Single-player / Premium cerrado

Producto terminado, sin necesidad de conexión permanente. Ejemplos: Celeste, Dead Cells.

Live Ops / Servicios en vivo

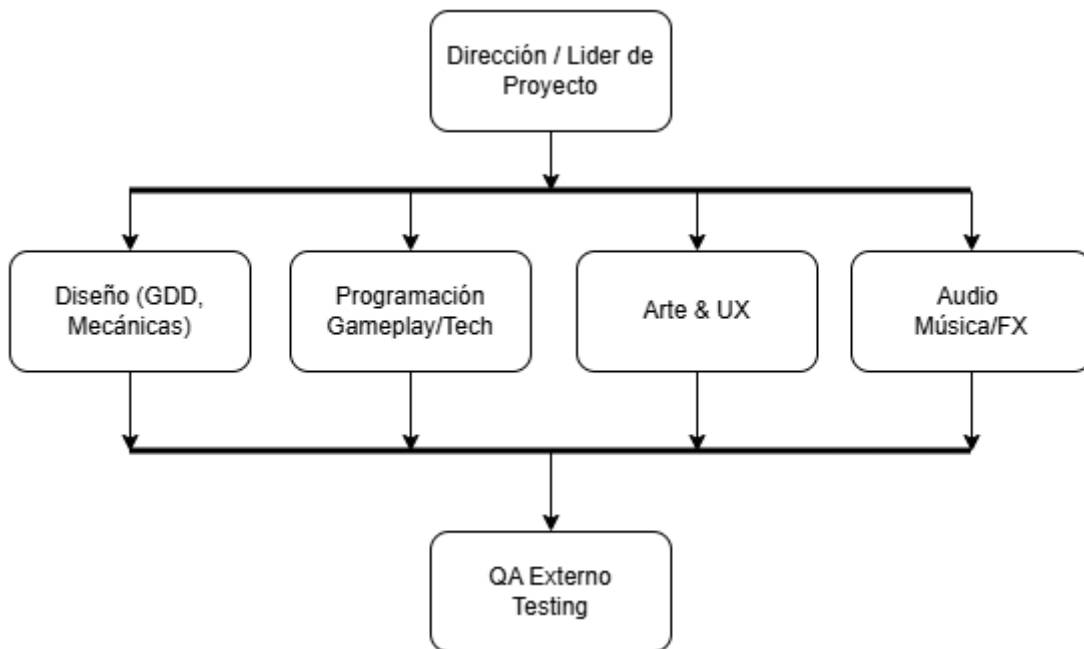
Juego persistente con actualizaciones, eventos, tiendas y analítica. Ejemplos: Hearthstone, Genshin Impact.

Empresas Tipo

1. Empresa tipo: Estudio Indie Premium (PC/Steam)

Pequeño equipo (1-10 personas) centrado en la creación de juegos premium para PC, publicados en Steam u otras plataformas digitales.

Estructura Organizativa



Funciones esenciales por departamento

Dirección / Líder de proyecto

Define la visión creativa y los objetivos del juego. Supervisa la planificación, presupuesto y calendario de producción. Coordina las áreas de diseño, programación y arte. Gestiona las relaciones con publishers o plataformas de distribución digital.

Diseño (GDD, mecánicas)

Crea el documento de diseño (GDD). Define las mecánicas, progresión, reglas, niveles y balance general. Supervisa la coherencia de la experiencia de usuario y el ritmo de juego.

Programación

Desarrolla el motor o utiliza uno existente (Godot, Unity, Unreal). Implementa sistemas de Gameplay, físicas, IA, UI y herramientas internas. Asegura el rendimiento y la optimización técnica.

Arte y UX

Diseña el estilo visual, modelado, texturas, animaciones y efectos. Desarrolla interfaces claras e intuitivas. Colabora con diseño para asegurar legibilidad y coherencia artística.

Audio Música/FX

Crea la identidad sonora del juego. Produce y mezcla música, efectos y ambientaciones. Colabora con programación para la integración de audio dinámico.

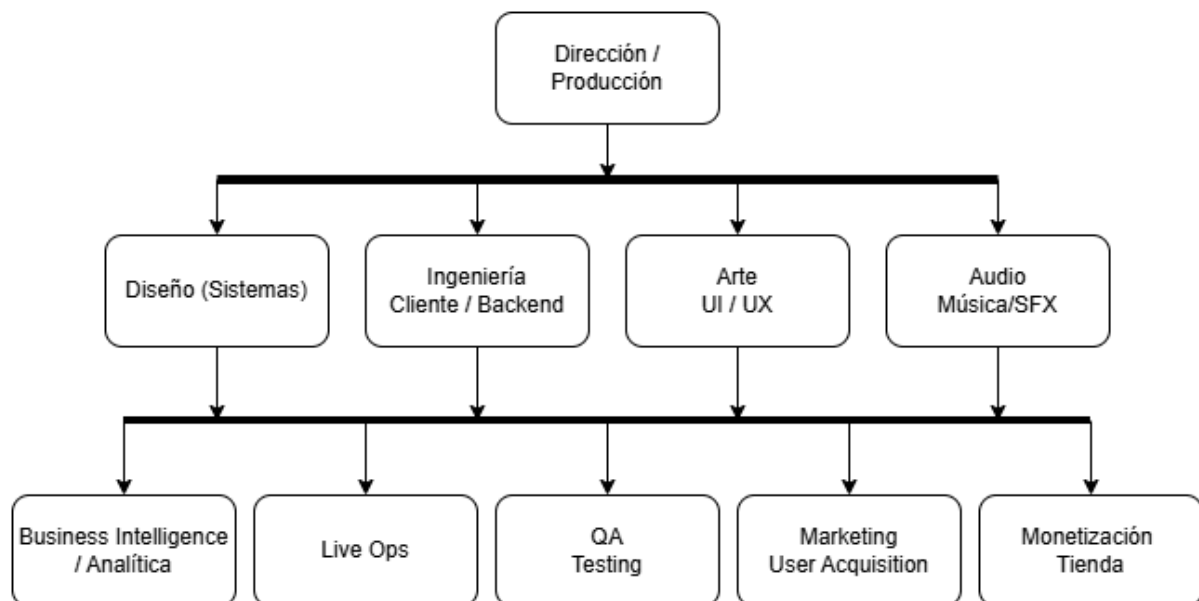
QA y Testing Externo

Realiza pruebas funcionales, de estabilidad y jugabilidad. Reporta errores y evalúa la experiencia de usuario. Verifica compatibilidad y rendimiento en los dispositivos objetivo.

2. Empresa tipo: Estudio móvil Free-to-Play (F2P) con Live Ops

Equipo mediano (10-40 personas) orientado a F2P. Ciclos de sprints con entregas frecuentes, fuerte dependencia de métricas y operaciones en vivo (eventos, pases de temporada).

Estructura Organizativa



Funciones esenciales por departamento

Dirección / Producción

Define la estrategia del producto, roadmap y prioridades. Supervisa la ejecución de sprints y actualizaciones. Coordina equipos multidisciplinarios y gestiona métricas clave (retención, monetización).

Diseño de Sistemas

Diseña la estructura del juego (niveles, progresión, recompensas, monetización). Crea bucles de retención (retos diarios, logros, eventos). Ajusta balance y tiempos de juego.

Ingeniería Cliente Backend

Desarrolla la aplicación móvil y el servidor. Gestiona autenticación, perfiles, sincronización en la nube y seguridad de datos. Implementa APIs, escalabilidad y optimización de red.

Arte UX/UI

Diseña interfaces adaptadas a móvil (toque, tamaño, orientación). Garantiza usabilidad y consistencia visual. Produce assets optimizados para distintas resoluciones.

Audio (Música/SFX)

Crea música dinámica, sonidos de interfaz, notificaciones y efectos contextuales. Supervisa niveles de audio y optimización.

Business Intelligence / Analítica

Registra y analiza métricas de usuario (retención, ingresos, conversiones). Elabora informes para la toma de decisiones sobre balance, diseño y marketing.

Live Ops

Planifica y ejecuta eventos temporales, misiones y temporadas. Actualiza contenido sin necesidad de versión nueva del cliente. Mantiene la actividad constante en la comunidad.

QA Testing

Realiza testing funcional, de compatibilidad entre dispositivos y redes. Automatiza pruebas críticas y coordina test previos a cada actualización.

Marketing y UA

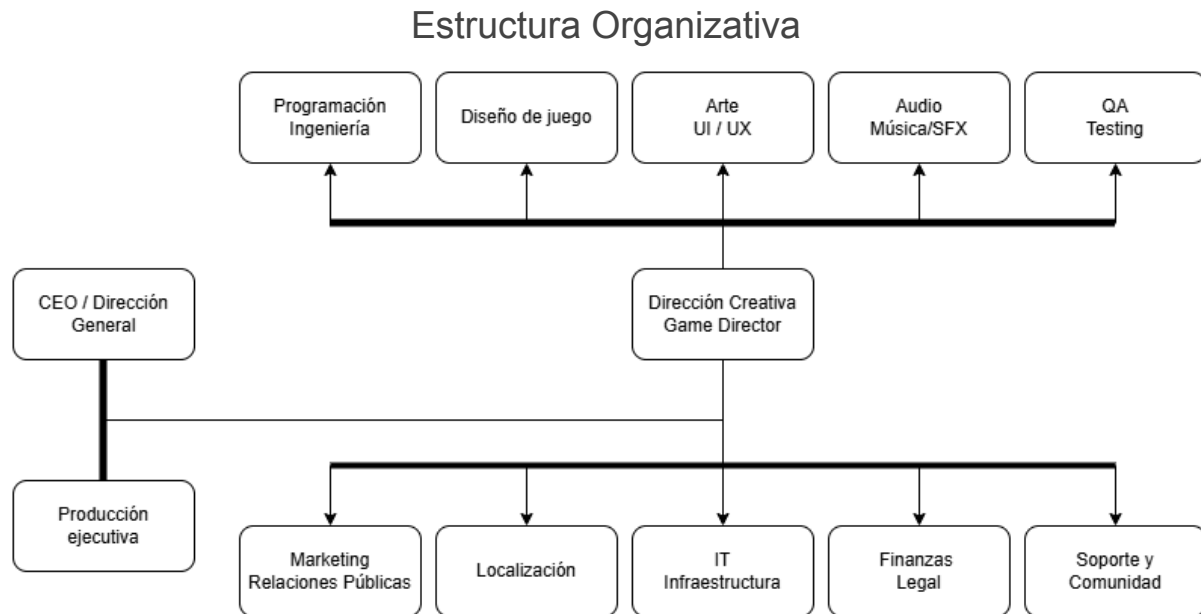
Gestiona campañas de adquisición de usuarios, anuncios, influencers y ASO (App Store Optimization). Mide retorno de inversión (ROAS).

Monetización / Tienda

Gestiona catálogos de ítems, precios, promociones y bundles. Realiza test A/B. Controla cumplimiento de políticas de Google Play/App Store.

3. Empresa tipo: Estudio AAA

Corporación con múltiples estudios internos o asociados. Estructura formal, milestones estrictos, certificaciones por plataforma, marketing global y grandes operaciones de QA/Localización.



Funciones esenciales por departamento

CEO / Dirección General

Define la estrategia global, asigna presupuestos y aprueba proyectos. Supervisa todos los departamentos.

Producción Ejecutiva

Coordina el desarrollo de cada juego. Gestiona plazos, recursos y objetivos de calidad.

Dirección Creativa / Game Director

Define la visión creativa y narrativa del juego. Supervisa el diseño, el tono artístico y la coherencia global.

Programación / Ingeniería

Implementa los sistemas de juego, físicas, IA, UI, red, etc. Desarrolla herramientas internas y asegura compatibilidad multiplataforma.

Diseño de juego

Desarrolla niveles, misiones, balance y narrativa interactiva. Colabora con QA para probar el flujo de juego.

Arte UI / UX

Crea modelos, texturas, entornos, efectos visuales, iluminación y cinemáticas. A veces hay subequipos por área (personajes, entornos, VFX).

Audio Música/SFX

Crea música, efectos y voces. Colabora con Localización y Diseño para sincronización de audio.

QA Testing

Ejecuta pruebas funcionales y de certificación (TRC/TCR). Verifica builds, bugs y compatibilidad.

Marketing

Planifica campañas publicitarias internacionales, trailers, presencia en ferias y prensa. Coordina influencers y redes sociales.

Localización

Traduce y adapta contenido a diferentes idiomas. Supervisa pruebas lingüísticas y coherencia de terminología.

IT Infraestructura

Mantiene servidores, bases de datos, control de versiones, infraestructura de red y almacenamiento.

Finanzas / Legal

Gestiona presupuestos, contratos, derechos de autor, clasificación PEGI/ESRB y licencias de terceros.

Soporte y Comunidad

Gestiona incidencias y reclamaciones. Proporciona asistencia postventa y seguimiento de bugs.

Implicaciones para mi aplicación

El proyecto Deck & Dagger: Goblin Hunt encaja plenamente con la filosofía y el funcionamiento de un estudio Indie Premium. Se caracteriza por un desarrollo individual con alta autonomía creativa, recursos ajustados y un enfoque centrado en la jugabilidad y la identidad artística más que en la monetización. El desarrollador asume múltiples roles (dirección, diseño, programación, arte y testeo) siguiendo una estructura horizontal típica del entorno indie, donde las decisiones son ágiles y las herramientas elegidas (Godot 4.5.1 y SQLite) priorizan eficiencia y bajo coste. La metodología aplicada es iterativa, basada en prototipos, pruebas continuas y refinamiento progresivo, replicando el ciclo real de producción de los estudios independientes.

1.2. Estudio de Mercado

1. Análisis de necesidades

Revisando el Anuario 2024 de AEVI:

- En 2024 la industria del videojuego en España facturó 2408 millones de euros, lo que representa un crecimiento moderado del 3% respecto a 2023.
- El número de personas que juegan habitualmente videojuegos en España asciende a 22,1 millones, lo que equivale aproximadamente al 45 % de la población.
- El mercado físico retrocede: las ventas físicas de videojuegos sumaron 850 millones de euros en 2024, con una disminución del aproximadamente 18,11% respecto a 2023. Mientras tanto, el mercado online creció un 19,75%, hasta 1.558 millones de euros.

A partir de estos datos podemos distinguir varias carencias/ineficiencias relevantes:

1. Carencia de experiencias adaptadas a nuevos usuarios.

Dado que el incremento del mercado es sólo del 3% en facturación, pese al aumento de usuarios, puede haber una brecha entre cantidad de jugadores y calidad de los productos ofrecidos.

2. Desplazamiento hacia distribución digital / online.

La caída fuerte del mercado físico (-18,11%) y la subida del online (+19,75%) indican que la forma de consumo está cambiando. Este cambio genera una ineficiencia operativa en cuanto a que muchos desarrollos podrían no estar optimizados para dispositivos móviles de gamas más bajas.

3. Falta de narrativa/localización dirigida al mercado hispanohablante.

Aunque los datos muestran un amplio número de jugadores en España, no se analizan en profundidad en el informe la integración cultural o localización, lo que podría suponer una carencia importante ya que la traducción al español a veces no contempla variaciones entre países hispanohablantes. Además de pocos juegos con doblaje castellano.

4. Desarrollo independiente con recursos limitados.

El Anuario subraya la necesidad de más talento y de incentivar el desarrollo nacional. Esto implica una carencia operativa: no hay suficientes herramientas sencillas y eficientes para estudios pequeños o unipersonales que quieren desarrollar juegos de calidad con recursos limitados, como sería mi caso.

5. Modelos de negocio saturados y necesidad de diferenciación.

El crecimiento moderado del sector a pesar de los grandes números indica que la competencia es alta y que los jugadores pueden estar menos dispuestos a títulos convencionales o poco innovadores. Esto revela una carencia de juegos "Premium" accesibles, sin microtransacciones agresivas.

2. Evaluación de la demanda y la competencia

El Anuario de la AEVI indica que el mercado español de videojuegos facturó más de 2.408 millones de euros en 2024, con un crecimiento del 3% respecto al año anterior. Además, la comunidad de jugadores en España alcanza los 22,1 millones de personas, aproximadamente el 45% de la población. Estas cifras muestran claramente una demanda significativa y en expansión en el mercado español.

Soluciones existentes en el ámbito “deck-builder / roguelike de cartas”

Aunque no hay un informe exhaustivo que liste todos los títulos de España en este nicho, a nivel global el género Deckbuilder-roguelike está muy presente con títulos consolidados como Slay the Spire, Pirates Outlaws, etc., que también están disponibles en España.

Fortalezas:

- Gran comunidad internacional y marketing más fuerte.
- Pulido técnico y contenido abundante.
- Reconocimiento de marca.

Debilidades:

- Alta competencia, muchos títulos similares, saturación de género.
- Modelos de negocio con microtransacciones, lo que genera rechazo en jugadores que buscan modelos “premium”.
- Poca adaptación/localización específica al mercado hispanohablante.
- Requisitos de hardware o afinamiento técnico que pueden excluir dispositivos de gama baja.

Evaluación del espacio para mi solución

La demanda es grande y relevante en España: millones de jugadores, crecimiento digital (ventas online). La evaluación muestra que el mercado español tiene una demanda considerable y que las soluciones actuales, aunque potentes, dejan espacios de oportunidad. La solución puede entrar en ese hueco ofreciendo una alternativa diferenciada. Esto muestra la viabilidad de lanzar un juego como Deck & Dagger orientado al mercado hispanohablante, con modelo indie premium y optimización para dispositivos variados.

3. Análisis DAFO de la aplicación

Factores internos

Fortalezas

1. Uso de tecnologías libres y multiplataforma.

El desarrollo con Godot 4.5.1 y base de datos SQLite reduce costes de licencia, facilita la portabilidad (Windows, Android) y garantiza independencia tecnológica, algo especialmente ventajoso para un proyecto unipersonal o de pequeño estudio.

2. Modelo de negocio “Indie Premium”

A diferencia de otros títulos del género, Deck & Dagger adopta un modelo de pago único sin microtransacciones.

3. Propuesta diferenciada en idioma y temática.

El juego está totalmente localizado al español y se inspira en una ambientación con estética propia buscando una temática diferente a otros deckbuilders.

4. Estructura técnica ligera y rendimiento optimizado.

Al no requerir conexión permanente ni servidores externos, puede ejecutarse en equipos modestos y dispositivos móviles, ampliando su alcance sin depender de infraestructura de red.

5. Equipo de testeo.

Multitud de compañeros interesados en probar el juego una vez este finalizado actuará como equipo de testeo para localizar bugs, errores visuales, etc.

Debilidades

1. Recursos limitados.

Al tratarse de un desarrollo individual, existen limitaciones en áreas como arte, sonido, marketing comparadas con estudios profesionales.

2. Ausencia de visibilidad inicial.

La falta de una comunidad previa o presupuesto de promoción dificulta el posicionamiento en tiendas digitales (Steam, GooglePlay).

3. Carga de tareas concentrada en una sola persona.

Las fases de diseño, programación, arte y documentación dependen de un único desarrollador, lo que incrementa el riesgo de retrasos y sobrecarga.

4. Necesidad de maduración de marca personal.

El proyecto no parte de una identidad reconocida, por lo que requerirá construir reputación y confianza en redes y foros especializados.

Factores externos

Oportunidades

1. Crecimiento sostenido del sector español del videojuego.

Según el Anuario 2024 de AEVI, la industria alcanzó una facturación record de 2.408 millones de euros y más de 22 millones de jugadores, con un incremento del 19,75% de ventas online. Esto refleja un entorno propicio para la distribución independiente online.

2. Interés social y regulatorio en modelos éticos.

La Dirección General de Consumo y la Comisión Europea vigilan las microtransacciones, lo que impulsa la preferencia por títulos sin sistemas de pagos abusivos.

3. Expansión del desarrollo indie nacional.

Existen más de 500 estudios independientes en España con comunidades activas en foros, ferias y eventos como IndieDevDay o GamePolis. Este ecosistema favorece la colaboración y visibilidad de proyectos individuales.

4. Auge del género deckbuilder roguelike.

Medios españoles como Pablo F.Iglesias y Meristation destacan el auge de los juegos de cartas con progresión roguelike y su expansión en el mercado digital. Este interés creciente ofrece un contexto favorable para nuevas propuestas.

Amenazas

1. Alta competencia internacional

El género está saturado por títulos exitosos como Slay the Spire, lo que dificulta destacar sin una propuesta visual o mecánica muy sólida.

2. Visibilidad limitada en tiendas digitales.

Las plataformas como Steam o Google Play priorizan productos con inversión publicitaria o valoraciones previas, lo que puede obstaculizar la llegada a nuevos jugadores.

3. Cambios en las tendencias de consumo.

La popularidad de géneros multijugador y juegos como servicio podría reducir el interés por experiencias roguelike offline a medio plazo.

4. Riesgos técnicos y de compatibilidad.

La fragmentación de dispositivos Android y la rápida evolución de hardware pueden requerir mantenimiento adicional si el proyecto se amplía comercialmente.

4. Propuesta y definición del tipo de proyecto

Deck & Dagger: Goblin Hunt es un videojuego de cartas tipo deckbuilder roguelike desarrollado con Godot 4.5.1 y SQLite, concebido como una aplicación Indie Premium de pago único, sin conexión obligatoria ni microtransacciones.

Su visión es ofrecer una experiencia accesible, estratégica y completamente en español (si me da tiempo también en inglés), centrada en la progresión por partidas (runs) y la mejora continua del mazo de cartas mediante recompensas y desbloques locales.

El alcance del proyecto abarca el desarrollo multiplataforma (PC y Android), con un diseño técnico ligero y modular que permite su ampliación futura, y un enfoque artístico propio que refuerza la identidad visual del producto.

Esta propuesta es viable y rentable: se apoya en herramientas libres, requiere baja inversión y responde directamente a las oportunidades detectadas en el mercado español (crecimiento digital, interés por juegos sin micropagos y auge del desarrollo indie nacional) según el Anuario 2024 de AEVI.

Por su coherencia con las tendencias del sector, su adecuación a los recursos disponibles y su capacidad para cubrir un nicho poco atendido, Deck & Dagger: Goblin Hunt representa una propuesta óptima, sostenida y alineada con las demandas actuales del mercado de los videojuegos en España.

Bibliografía

Asociación Española de Videojuegos

<https://aevi.org.es/>

<https://aevi.org.es/wp-content/uploads/2025/05/VDIGITAL-ANUARIO2024-comprimido.pdf>

<https://www.pantallasamigas.net/industria-videojuego-espana-anuario-2024-asociacion-espanola-videojuegos-aevi>

La época dorada de los roguelike basados en deck builders

<https://www.pabloyglesias.com/roguelike-deckbuilder-slay-spire-banners-ruin>

Meristation

https://as.com/meristation/2018/03/13/avances/1520924400_173887.html

<https://as.com/meristation/noticias/asi-es-hermetica-el-nuevo-roguelike-deckbuilder-espanol-sobre-una-toledo-de-ensueno-n>

1.3. Innovación, dificultad e investigación

1. Innovación

Deck & Dagger: Goblin Hunt destaca por integrar en un solo proyecto una propuesta técnica, artística y de experiencia de usuario original dentro del género deckbuilder roguelike. Aunque se inspira en títulos como Slay the Spire o Wildfrost, su desarrollo en Godot 4.5.1 aporta una visión innovadora tanto en el proceso como en el resultado final.

A nivel técnico, el proyecto se construye completamente sobre software libre multiplataforma, demostrando que es posible desarrollar un deckbuilder sólido fuera de los motores comerciales más utilizados (Unity, Unreal). Se integra SQLite como base de datos local para gestionar la persistencia de datos (mazos, progreso, cartas desbloqueadas y estadísticas), algo poco habitual en este tipo de juegos. Además, cada subsistema (combate, tienda, eventos, progresión, mapa) se implementa como nodos independientes reutilizables, aplicando una arquitectura modular que facilita la escalabilidad y el mantenimiento.

Desde el punto de vista artístico, el proyecto apuesta por una estética artesanal y original, con ambientación dark-fantasy humorística centrada en goblins, ilustraciones creadas por el propio autor y una banda sonora y efectos sonoros originales, reforzando la identidad del juego. La interfaz se diseña con un enfoque minimalista y adaptado a diferentes resoluciones, optimizando la experiencia tanto en PC como en dispositivos móviles.

En cuanto a la experiencia de jugador, la innovación se centra en una progresión accesible y rejugable. Se incluye un sistema de nivel global con desbloqueo de cartas, un mapa de eventos dinámico entre combates, y decisiones con impacto sobre la run, equilibrando estrategia, azar y narrativa ligera. El juego está diseñado para ofrecer partidas breves pero intensas, optimizadas para sesiones rápidas y una curva aprendizaje fluida.

2. Dificultad

El desarrollo de Deck & Dagger: Goblin Hunt presenta una dificultad técnica destacable, especialmente por la complejidad de los sistemas que intervienen en un deckbuilder.

El principal reto reside en el sistema de combate por turnos basado en cartas, que requiere una gestión precisa del flujo de estados (inicio de turno, selección de cartas, ejecución de efectos, y transición entre jugador y enemigos). Implementar este sistema de Godot 4.5.1 supone dominar el sistema de señales y nodos, así como la programación estructurada de efectos acumulativos, daño, defensa, energía y condiciones especiales.

Otro componente complejo es la persistencia de datos mediante SQLite, que implica diseñar tablas para almacenar progreso, estadísticas y desbloques, así como sincronizar la información entre partidas.

El diseño del mapa de progresión y eventos también requiere implementar una estructura de nodos conectados dinámicamente, que se generan de forma procedural y condicionan los encuentros o recompensas disponibles.

A nivel de interfaz, se enfrenta el desafío de construir una UI dinámica e intuitiva, que gestione el arrastre de cartas, animaciones y retroalimentación visual sin comprometer el rendimiento.

El proyecto, desarrollado de forma individual, añade además la dificultad de coordinar todas las áreas (programación, diseño visual, sonido y testeo) dentro de un flujo de trabajo unipersonal, exigiendo una planificación técnica muy precisa.

3. Trabajo de investigación

El proyecto ha requerido un amplio proceso de investigación y autoaprendizaje en varias áreas clave:

- Godot 4.5.1 y GDScript: se parte completamente desde cero en el uso del motor (inicio de Octubre junto a la primera propuesta). Se ha estudiado su arquitectura basada en nodos, el sistema de signals, la creación de autoloads y la gestión de escenas para desarrollar un flujo modular y eficiente.
- Integración de SQLite: se investiga cómo conectar bases de datos locales desde Godot, diseñando tablas y consultas SQL optimizadas para guardar progreso y estadísticas de usuario.
- Diseño de interfaz en Godot: aprendizaje de Nodos tipo Control, estilos visuales, fuentes dinámicas, adaptabilidad de resolución y jerarquía de elementos para obtener una interfaz clara y responsiva.
- Metodología de desarrollo ágil: se aplican principios de Scrum, con planificación en sprints semanales y tablero Kanban para documentar avances.
- Ecosistema audiovisual: se exploran herramientas de edición y creación de audio y arte propias, con el fin de generar materiales originales que aporten una identidad única al producto.

1.4. Ayudas y subvenciones

1. Ministerio de Cultura

Ayudas públicas para la promoción del sector del videojuego, del pódcast y de otras formas de creación digital (2025)

Organismo Convocante

Subsecretaría de Cultura, dependiente del Ministerio de Cultura y Deporte de España (Gobierno de España).

Tipo de ayuda

Subvención a fondo perdido, en régimen de concurrencia competitiva.

Objeto/Finalidad Principal

Apoyar al ecosistema del videojuego, del pódcast y de otras formas de creación digital, en el marco del Plan de Recuperación, Transformación y Resiliencia (PRTR), lo que incluye preproducción, producción y postproducción de videojuegos con carácter o interés cultural

Requisitos del Beneficiario

Que el proyecto este dentro del ámbito del videojuego/podcast/creación digital y tenga interés cultural.

Las actividades deben desarrollarse entre el 1 de enero y el 31 de diciembre de 2025 y los justificantes de gasto pagados deben estar dentro de plazo.

Haber presentado la solicitud en los plazos establecidos.

Monto/Porcentaje de Financiación Aproximado

Hasta 80.000€ por proyecto; la ayuda no puede superar el 80% del coste total subvencionable.

Dotación total de la convocatoria en 2025: 5.000.000 €.

Estado de la convocatoria

Finalizada

Aplicación al proyecto

Es aplicable porque Deck & Dagger: Goblin Hunt es un videojuego original de creación propia con valor artístico, narrativo y musical, desarrollado íntegramente en España.

La ayuda está dirigida a proyectos culturales y creativos, por lo que encaja perfectamente al tratarse de una producción independiente centrada en la jugabilidad, ambientación y dirección artística.

Además, permite financiar fases como producción, localización y promoción, que son esenciales para este proyecto indie premium.

2. NEOTEC Programa 2025

Centro para el Desarrollo Tecnológico e Innovación (CDTI)

Organismo

CDTI (Organismo estatal dependiente del Ministerio de Ciencia e Innovación)

Tipo de ayuda

Subvención (concurrencia competitiva) a fondo perdido para empresas de base tecnológica.

Objeto/Finalidad Principal

Apoyar la creación y consolidación de empresas de base tecnológica (EBT) cuya actividad se base en el uso de tecnologías o conocimientos desarrollados a partir de la actividad investigadora, y donde la estrategia de negocio se base en el desarrollo de tecnología propia.

Requisitos del Beneficiario

Empresa innovadora ("base tecnológica) constituida y que su actividad principal sea tecnológica/innovadora.

Antigüedad máxima de 3 años desde la constitución al cierre del plazo de solicitud.

Capital social mínimo: al menos 20.000€ o capital social + prima de emisión $\geq$ 20.000€.

No estar cotizando en bolsa.

No haber distribuido beneficios, no haber surgido de una operación de concentración, etc.

Monto/Porcentaje de Financiación Aproximado

Hasta 70% del presupuesto elegible como ayuda normal; en proyectos que incorporen al menos un doctor y cumplan otros requisitos, hasta 85%, con un máximo de 325.000€.

Estado de la convocatoria

Finalizada.. Plazo de solicitud del 12 de Mayo al 12 de junio de 2025.

Aplicación al proyecto

Es aplicable porque el proyecto tiene un componente tecnológico innovador: el uso de Godot 4.5.1 y SQLite para implementar mecánicas de generación de eventos, progresión y equilibrio de cartas.

Estas características lo convierten en un producto con potencial para evolucionar hacia una herramienta tecnológica o librería reutilizable, cumpliendo con el requisito de empresa de base tecnológica que busca esta ayuda.

3. Préstamo participativo (ENISA)

Líneas de financiación - Empresa Nacional de Innovación

Organismo

ENISA, entidad pública bajo el Ministerio de Industria, Comercio y Turismo de España.

Tipo de ayuda

Préstamo participativo (financiación no dilutiva; lleva intereses vinculados a resultado empresa).

Objeto/Finalidad Principal

Financiar startups y pymes innovadoras con modelos de negocio escalables, que requieren financiación para crecimiento o consolidación, sin ceder participación, y con un fuerte componente de innovación.

Requisitos del Beneficiario

Sociedad mercantil con domicilio social en España.

Startups y Pymes: Importe mínimo 25.000€.

“Emprendedoras digitales”, “Audiovisual e Industrias Culturales y Creativas” u otras verticales, condiciones adicionales (por ejemplo, liderazgo femenino).

Modelos de negocio innovador, viabilidad técnica y económica demostrada.

Fondos propios de la sociedad al menos equivalente al préstamo solicitado (cofinanciación).

Sin avales personales exigidos, ni garantías reales en muchos casos.

Monto/Porcentaje de Financiación Aproximado

Préstamos desde 25.000€ hasta 1.500.000€ (según la fase).

Al no ser subvención no se habla de porcentajes de cobertura, se concede el préstamo bajo las condiciones de amortización, carencia, etc.

Estado de la convocatoria

Activa.

Aplicación al proyecto

Es aplicable porque permite financiar startups o proyectos independientes con modelos de negocio innovadores y viables, sin exigir avales personales.

Deck & Dagger podría beneficiarse para cubrir marketing, distribución y certificaciones multiplataforma, lo que encaja con el objetivo de ENISA de apoyar el crecimiento y consolidación de proyectos tecnológicos y creativos.

4. Instituto Aragonés de Fomento

Ayudas a empresas para impulsar proyectos de innovación a través de la iniciativa TechFabLab - Convocatoria 2025 (Aragón)

Organismo

Instituto Aragonés de Fomento (IAF), en el marco de la iniciativa TechFabLab, Comunidad Autónoma de Aragón.

Tipo de ayuda

Subvención a fondo perdido (regulación de “minimis”).

Objeto/Finalidad Principal

Impulsar proyectos de innovación de startups y pymes aragonesas, a través del proyecto TechFabLab (orientado a tecnologías facilitadoras esenciales, innovación abierta, digitalización, etc).

Requisitos del beneficiario

Empresa ubicada en Aragón(Startup o Pyme).

Pymes de menos de 10 años.

Proyecto que implique innovación, tecnología o digitalización.

Coste mínimo del proyecto: al menos 20.000€.

Monto/Porcentaje de financiación aproximado

Ayuda máxima por proyecto: 30.000€.

Porcentaje: hasta el 80% del coste para startups; hasta 50% para resto de pymes.

Estado de la convocatoria

Finalizada. Del 10 de junio hasta el 24 de Junio de 2025. 10 días hábiles.

Aplicación al proyecto

Es aplicable porque el desarrollo se realiza desde Zaragoza y el proyecto introduce innovación digital y tecnológica en el ámbito cultural y del entretenimiento.

Esta línea está pensada para pymes y startups aragonesas que aplican tecnología y fomentan la digitalización, por lo que se ajusta perfectamente al perfil de estudio independiente que desarrolla un videojuego propio con base tecnológica.

Bibliografía

Ministerio de Cultura

<https://www.cultura.gob.es/cultura/industriasculturales/sc/becas-ayudas-subvenciones/promocion-videojuego-creacion-digital.html>

<https://cultura.sede.gob.es/procedimientos/portada/ida/2584/idp/19512>

<https://planderecuperacion.gob.es/noticias/Conoce-ayudas-promocioo-sector-videojuego-podcast-creacion-digital-2025-prtr>

NEOTEC Programa 2025

<https://www.cdti.es/ayudas/ayudas-neotec-2025>

<https://financiacioneinvestigacion.com/lp/convocatoria-neotec-2025/>

Préstamo participativo (ENISA)

<https://www.enisa.es/es/financia-tu-empresa/faqs>

<https://www.kleohub.com/recursos/guias/guia-enisa2025>

Instituto Aragonés de Fomento

<https://www.aragon.es/tramitador/-/tramite/ayudas-a-empresas-para-impulsar-proyectos-de-innovacion-a-traves-de-la-iniciativa-techfablab/convocatoria-2025>

<https://techfablab.es/lanzamiento-de-la-convocatoria-de-ayudas-de-innovacion-a-startups-y-pymes-enmarcado-en-el-proyecto-tech-fablab-en-aragon-2>

1.5. Especificación de requisitos

1. Requisitos funcionales (RF)

1.1. Interfaz y navegación

- RF-01: El sistema mostrará un menú principal con las opciones Nueva partida, Continuar, Biblioteca, Perfil y Salir.
- RF-02: El usuario podrá navegar entre pantallas mediante botones claramente identificables.
- RF-03: El sistema mostrará una pantalla de perfil del jugador con la información básica, logros y estadísticas.
- RF-04: La aplicación mostrará una pantalla de biblioteca de cartas que permitirá visualizar las cartas desbloqueadas.

1.2. Jugabilidad y Sistema de Combate

- RF-05: El juego implementará un sistema de combate por turnos entre el jugador y uno o más enemigos.
- RF-06: El jugador usará cartas de ataque, defensa o habilidad durante su turno.
- RF-07: El sistema mostrará las intenciones de los enemigos antes de actuar.
- RF-08: El combate finalizará cuando el enemigo o el jugador queden sin puntos de vida.
- RF-09: Al ganar un combate, el jugador podrá obtener recompensas (oro, nuevas cartas y puntos de experiencia).

1.3. Sistema de progresión

- RF-10: El sistema otorgará experiencia (XP) al jugador al finalizar cada combate.
- RF-11: El jugador podrá desbloquear nuevas cartas al alcanzar determinados niveles de experiencia.
- RF-12: El sistema mostrará una pantalla de logros que refleje los hitos alcanzados.
- RF-13: El sistema registrará el progreso del jugador (nivel, cartas desbloqueadas, mapa alcanzado).

1.4. Mapa de nodos y navegación

- RF-14: El jugador podrá desplazarse por un mapa de nodos que represente los diferentes encuentros (combates, eventos, tiendas y jefes).
- RF-15: El sistema generará los nodos de forma semiestructurada o aleatoria, asegurando variedad de caminos.
- RF-16: Cada nodo activará un tipo de evento: combate, evento, tienda, jefe.

1.5 Tienda y eventos

- RF-17: En la tienda, el jugador podrá comprar cartas utilizando el oro obtenido.
- RF-18: Los eventos ofrecerán decisiones de texto con diferentes consecuencias.

1.6 Persistencia de datos

- RF-19: El sistema almacenará en SQLite la información del jugador.
- RF-20: Al iniciar la aplicación, el sistema cargará los datos guardados y permitirá continuar la partida anterior.
- RF-21: La base de datos mantendrá integridad referencial entre tablas (jugador, progreso, estadísticas).

2. Requisitos No Funcionales (RNF)

2.1. Rendimiento y optimización

- RNF-01: El prototipo deberá ejecutarse con fluidez en equipos de gama media (mínimo CPU i3, 8GB RAM).
- RNF-02: Los tiempos de carga entre pantalla no superarán los 3 segundos.

2.2. Compatibilidad y portabilidad.

- RNF-03: La aplicación será compatible con Windows y Android..
- RNF-04: Se garantizará la portabilidad del proyecto mediante exportación desde Godot a Windows y Android.

2.3. Usabilidad y accesibilidad

- RNF-05: La interfaz será intuitiva y coherente, con la iconografía clara y tipología legible.
- RNF-06: Los textos y menús estarán disponibles en idioma español.
- RNF-07: Los botones tendrán tamaños adecuados para interacción con ratón o pantalla táctil.

2.4. Seguridad y fiabilidad

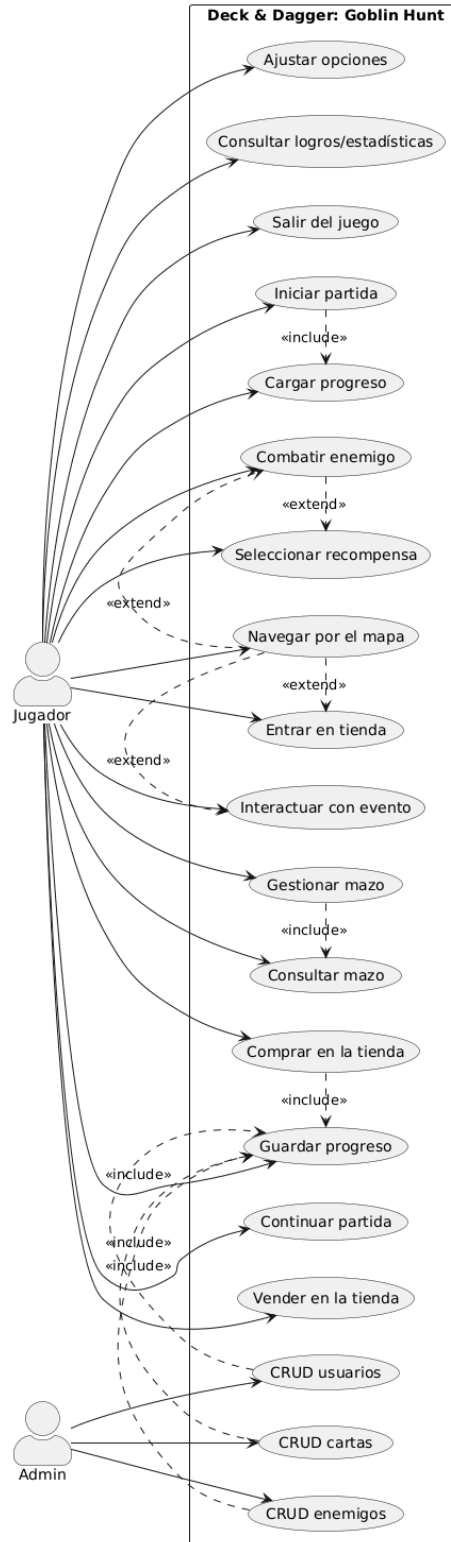
- RNF-08: EL sistema realizará guardado automático tras cada combate o evento importante.
- RNF-09: Los datos en SQLite estarán protegidos mediante validaciones internas par evitar corrupción o duplicidad.

2.5. Mantenimiento y escalabilidad

- RNF-10: El código estará estructurado por escenas y scripts independientes en Godot, siguiendo principios de modularidad.
- RNF-11: Se documentarán las clases y funciones principales mediante comentarios en GDScript.
- RNF-12: El diseño permitirá ampliar fácilmente el número de cartas, enemigos o eventos sin alterar la arquitectura base.

1.6. Casos de Uso

1. Diagrama Casos de Uso



1.1 Actores

- Jugador: interactúa con el juego.
- Admin: gestiona la base de datos (cartas, enemigos y usuarios).

1.2 Casos de uso del Jugador

1.2.1 Menú Principal

CU-01 Iniciar partida

Actor: Jugador

Descripción: Permite comenzar nueva run desde el menú principal, generando el mapa inicial y el mazo base.

CU-02 Continuar partida

Actor: Jugador

Descripción: Permite comenzar nueva run desde el menú principal, generando el mapa inicial y el mazo base

CU-03 Ajustar opciones

Actor: Jugador

Descripción: Accede a la pantalla de opciones para configurar audio.

CU-04 Consultar logros/estadísticas

Actor: Jugador

Descripción: Muestra los logros e información de las partidas jugadas.

CU-05 Salir del juego

Actor: Jugador

Descripción: Cierra la aplicación.

1.2.2 Mapa

CU-10 Navegar mapa

Actor: Jugador

Descripción: Permite desplazarse entre los nodos del mapa: combate, evento, tienda...

CU-11 Entrar nodo combate

Actor: Jugador

Descripción: Selecciona un nodo de combate en el mapa para iniciar un enfrentamiento contra un enemigo.

CU-12 Entrar nodo evento

Actor: Jugador

Descripción: Selecciona un nodo de evento para acceder a un evento narrativo de toma de decisiones.

CU-13 Entrar nodo tienda

Actor: Jugador

Descripción: Entrar nodo tienda para abrir el interfaz de tienda

1.2.3 Combate

CU-20 Combatir enemigo

Actor: Jugador

Descripción: Resuelve combate por turnos entre jugador y uno o varios enemigos hasta lograr la victoria o ser derrotado.

CU-21 Seleccionar recompensa

Actor: Jugador

Descripción: Al salir victorioso de un combate se elige una carta entre varias opciones para agregar al mazo.

1.2.4 Evento y Tienda

CU-30 Participar en evento

Actor: Jugador

Descripción: Lee la descripción del evento y selecciona una de las decisiones.

CU-31 Comprar en tienda

Actor: Jugador

Descripción: Adquiere cartas de tienda gastando oro siempre que se cumplan los requisitos necesarios.

CU-32 Eliminar carta

Actor: Jugador

Descripción: Permite eliminar cartas del mazo gastando oro.

1.2.5 Mazo

CU-40 Ver mazo

Actor: Jugador

Descripción: Accede al mazo para poder ver las cartas.

1.2.6 Progreso

CU-50 Guardar progreso

Actor: Jugador

Descripción: Guarda el estado de la partida (mazo, mapa, vida, oro, etc.) en la base de datos SQLite.

CU-51 Cargar progreso

Actor: Jugador

Descripción: Recupera un guardado previo y restaura el estado de la run.

CU-52 Consultar estadísticas y logros

Actor: Jugador

Descripción: Muestra las estadísticas del jugador y logros desbloqueados.

1.3 Casos de Uso del Admin

CU-100 CRUD Cartas

Actor: Admin

Descripción: Crea, muestra, edita o elimina las cartas del juego.

CU-101 CRUD Enemigos

Actor: Admin

Descripción: Crea, muestra, edita o elimina los enemigos del juego.

CU-52 CRUD Gestion usuarios

Actor: Admin

Descripción: Lista Usuarios y cambia estado entre jugador o Admin. Elimina usuarios.

2. Casos de uso críticos

A continuación se describen de forma detallada los casos de uso críticos seleccionados, considerados esenciales para el funcionamiento del videojuego Deck & Dagger: Goblin Hunt. Estos casos representan el núcleo del ciclo de juego: iniciar una partida, combatir, obtener recompensas, mejorar el mazo en la tienda y resolver eventos del mapa.

2.1 CU-01 – Iniciar partida

Actor principal: Jugador

Descripción: Este caso de uso permite al jugador comenzar una nueva partida o continuar una ya existente. El sistema inicializa todos los elementos necesarios para el desarrollo de la run, incluyendo el mapa, el mazo base y los recursos iniciales del jugador.

Precondiciones:

- El videojuego está en ejecución y se muestra el Menú principal.
- La base de datos SQLite está operativa y accesible.
- Existen los datos necesarios para inicializar una partida (mazo inicial, estructura del mapa, cartas y enemigos base).

Postcondiciones:

- Si el jugador inicia una nueva partida, el sistema genera un estado inicial completo (mazo base, mapa, vida, oro y reliquias iniciales).
- Si el jugador elige continuar partida, el sistema ha cargado correctamente la información de la run guardada.
- El jugador es trasladado a la Pantalla de mapa, quedando el juego listo para iniciar la exploración.

Flujo principal:

1. El jugador accede al Menú principal.
2. Selecciona la opción “Nueva partida” o “Continuar partida”.
3. El sistema comprueba si existen partidas guardadas en SQLite.
4. Si se selecciona Nueva partida:
 - 4.1. El sistema crea un nuevo estado de partida.
 - 4.2. Asigna el mazo base al jugador.
 - 4.3. Genera el mapa del acto correspondiente.
 - 4.4. Inicializa los recursos del jugador (vida, oro, reliquias).
5. Si se selecciona Continuar partida:
 - 5.1. El sistema lee el estado guardado en SQLite.
 - 5.2. Reconstruye la partida previa (mazo, vida, mapa, nodo actual).
6. El sistema muestra la Pantalla de mapa.
7. Fin del caso de uso.

Flujos alternativos / de excepción:

A1 – No existen partidas guardadas:- El sistema desactiva o bloquea la opción “Continuar partida”.

E1 – Error al acceder a datos (SQLite):- El sistema informa al jugador y ofrece reintentar o regresar al menú.

2.2 CU-20 – Combatir enemigo

Actor principal: Jugador

Descripción: Gestiona el sistema de combate por turnos entre el jugador y uno o varios enemigos. El jugador utiliza cartas de su mazo para realizar acciones ofensivas, defensivas o estratégicas hasta determinar el resultado del enfrentamiento.

Precondiciones:

- El jugador ha seleccionado un nodo de combate en el mapa.
- El sistema ha cargado los datos del enemigo (vida, patrones de ataque, recompensas).
- El mazo del jugador y las mecánicas de energía están correctamente inicializadas.

Postcondiciones:

- Si el jugador gana, el sistema genera las recompensas y activa el caso de uso CU-03 Seleccionar recompensa.
- Si el jugador pierde, la run finaliza y se muestra la pantalla de fin de partida.

Flujo principal:

1. El jugador entra a un nodo de combate desde el mapa.
2. El sistema muestra la Pantalla de combate, generando la mano inicial del jugador.
3. Turno del jugador:
 - 3.1. El jugador elige una carta de su mano.
 - 3.2. El sistema valida energía y objetivo.
 - 3.3. Se aplican los efectos de la carta (daño, bloqueo, estados, robo).
 - 3.4. Las cartas jugadas pasan al descarte.
 - 3.5. El jugador puede seguir jugando cartas o finalizar su turno.
4. Turno del enemigo:
 - 4.1. La IA selecciona una intención basada en su patrón.
 - 4.2. Aplica daño o estados al jugador.
 - 4.3. El sistema actualiza vida y estados.
5. El sistema comprueba la condición de victoria/derrota:
 - Si la vida del enemigo es menor o igual que 0, se produce la victoria.
 - Si la vida del jugador es menor o igual que 0, se produce la derrota.
 - En caso contrario, se inicia un nuevo turno de jugador volviendo al paso 3.
6. Si hay victoria, el sistema genera el botín del combate.
7. Se activa CU-21 – Seleccionar recompensa.
8. Fin del caso de uso.

Flujos alternativos / de excepción:

A1 – Energía insuficiente para jugar una carta:- Se muestra aviso y la carta no se ejecuta.

A2 – Carta con objetivo no válido:- El sistema informa y cancela la acción.

E1 – Error al aplicar efectos:- El sistema intenta continuar el combate sin comprometer la integridad de datos.

2.3 CU-21 – Seleccionar recompensa

Actor principal: Jugador

Descripción: Tras ganar un combate, el jugador puede elegir una recompensa entre varias opciones, generalmente cartas, oro. Esta selección influye directamente en la estrategia del mazo.

Precondiciones:

- El jugador ha ganado un combate (CU-20 – Combatir enemigo).
- El sistema ha generado un conjunto de recompensas válidas.

Postcondiciones:

- La carta o recompensa seleccionada se añade al mazo o inventario.
- Se actualizan oro y recursos si corresponde.
- El jugador regresa al Mapa.

Flujo principal:

1. El sistema muestra la Pantalla de recompensas.
2. El jugador selecciona una de las cartas u opciones disponibles.
3. El sistema:
 - 3.1. Añade la carta al mazo.
 - 3.2. Aplica las recompensas adicionales (oro, reliquias).
 - 3.3. Actualiza el estado del jugador en memoria.
4. El sistema cierra la pantalla de recompensa.
5. El jugador regresa al mapa.
6. Fin del caso de uso.

Flujos alternativos / de excepción:

A1 – El jugador elige “No añadir carta”:

- El mazo no cambia, pero se aplican recompensas secundarias.

E1 – Error al guardar cambios:

- Se intenta una segunda persistencia y, si falla, se informa al jugador.

2.4 CU-30 – Participar en evento

Actor principal: Jugador

Descripción: Permite al jugador interactuar con un evento narrativo o especial del mapa, eligiendo entre varias opciones que pueden generar efectos positivos, negativos o mixtos sobre su estado (vida, oro, cartas, etc.).

Precondiciones:

- El jugador ha seleccionado un nodo de evento en el mapa.- El evento ha sido cargado correctamente desde la base de datos o desde la configuración interna del juego.

Postcondiciones:

- Se aplican las consecuencias derivadas de la decisión tomada por el jugador.
- El jugador vuelve al mapa, con su estado actualizado en función del evento.

Flujo principal:

1. El jugador entra en el nodo de evento desde el mapa.
2. El sistema muestra la descripción narrativa del evento y su contexto.
3. El sistema presenta las opciones disponibles (por ejemplo, arriesgar vida a cambio de oro, sacrificar cartas a cambio de otra aleatoria, etc.).
4. El jugador selecciona una de las opciones mostradas.
5. El sistema evalúa la opción elegida y aplica sus consecuencias:
 - 5.1. Modificación de vida, oro, cartas, estados.
 - 5.2. Posible obtención de recompensas adicionales o penalizaciones.
6. El sistema actualiza el estado del jugador en memoria y, si procede, en la base de datos.
7. El sistema cierra la pantalla de evento.
8. El jugador vuelve al mapa.
9. Fin del caso de uso.

Flujos alternativos / de excepción:

A1 – Opción bloqueada por requisitos:- Si el jugador no cumple los requisitos de una opción (por ejemplo, no tiene suficiente vida o cartas a sacrificar), el sistema informa de que dicha opción no está disponible.

E1 – Error de persistencia:- El sistema intenta reapplicar los cambios o informa al jugador en caso de no poder guardar el nuevo estado de forma correcta

2.5 CU-31 – Comprar en la tienda

Actor principal: Jugador

Descripción: Permite al jugador adquirir nuevas cartas usando oro. Esta acción es fundamental para mejorar el mazo y aumentar la probabilidad de éxito en futuras batallas.

Precondiciones:

- El jugador ha seleccionado un nodo de tienda en el mapa.
- La tienda se ha generado correctamente con stock y precios.
- El jugador posee una cantidad determinada de oro.

Postcondiciones:

- Se actualiza el oro del jugador.
- Los ítems adquiridos se añaden al mazo o inventario.
- Se actualiza el stock de la tienda.

Flujo principal:

1. El jugador accede a la Pantalla de tienda.
2. El sistema muestra el catálogo disponible (cartas).
3. El jugador selecciona un ítem y pulsa Comprar.
4. El sistema valida:
 - Que el jugador tiene oro suficiente.
 - Que el ítem sigue disponible.
5. Si la compra es válida:
 - 5.1. Se descuenta el oro.
 - 5.2. Se añade la carta al jugador.
 - 5.3. Se actualiza el estado de la tienda (agotado / comprado).
6. El jugador puede seguir comprando o salir de la tienda.
7. Al salir, el sistema devuelve al jugador al mapa.
8. Fin del caso de uso.

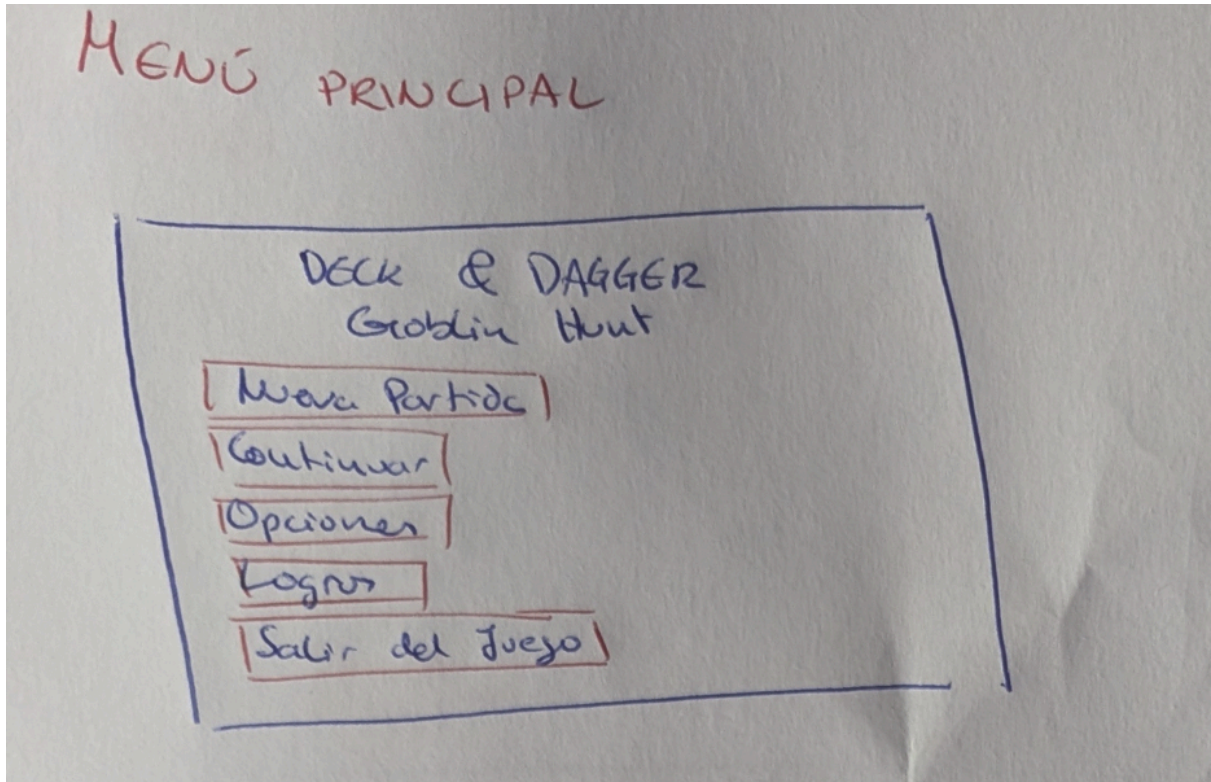
Flujos alternativos / de excepción:

- A1 – Oro insuficiente:- Se muestra mensaje de “Oro insuficiente” y no se realiza la compra.
- A2 – Eliminar carta del mazo (si existe este servicio):- El jugador selecciona una carta a eliminar y el sistema la retira del mazo.
- E1 – Error en persistencia:- Se mantiene el cambio en memoria y se reintenta la operación.

1.7. Bocetos iniciales

Diseño de Bocetos y 2. Interacción

1.1. Menú Principal



Título

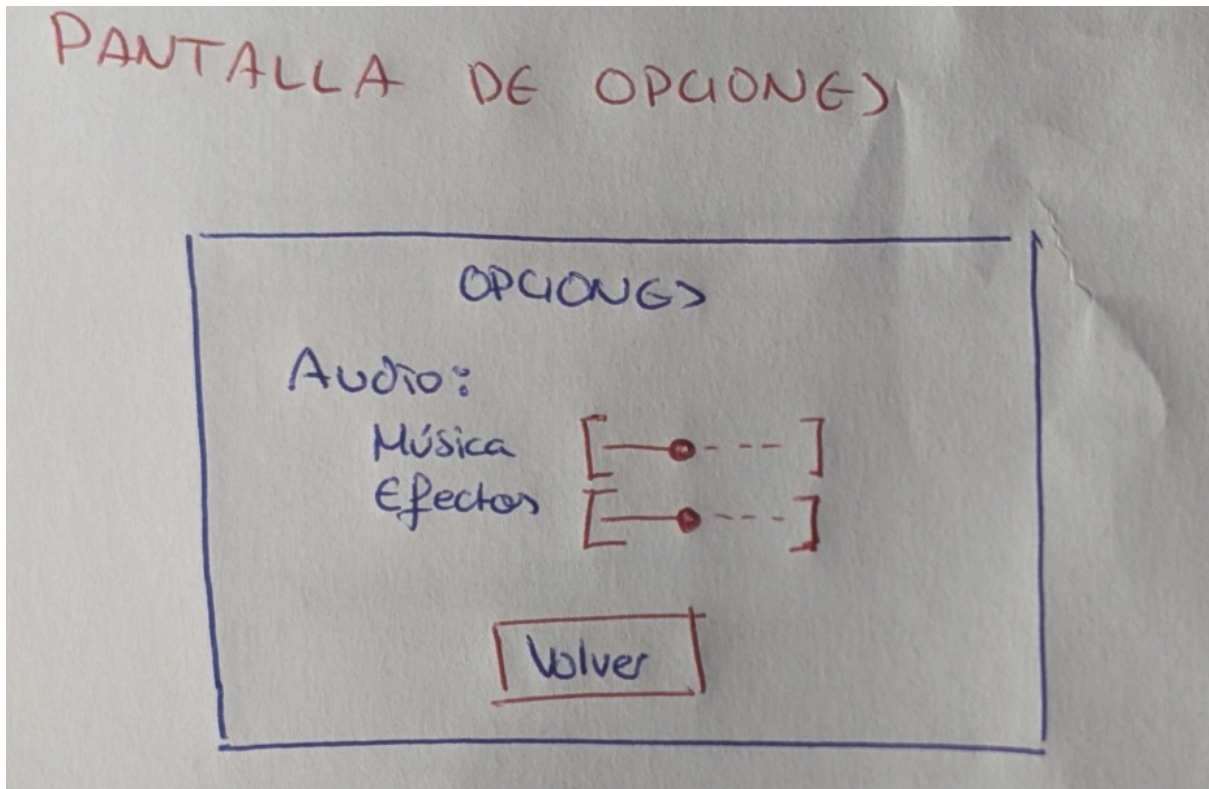
Botones:

- Nueva Partida
- Continuar
- Opciones
- Logros
- Salir del juego

2.1 Interacción menú principal

- Nueva partida → Pantalla de Mapa
- Continuar partida → Pantalla de Mapa
- Opciones → Pantalla Opciones
- Logros/Estadísticas → Pantalla Logros
- Salir → Cierra la aplicación

1.2 Opciones



Título

Sliders:

- Música
- Efectos

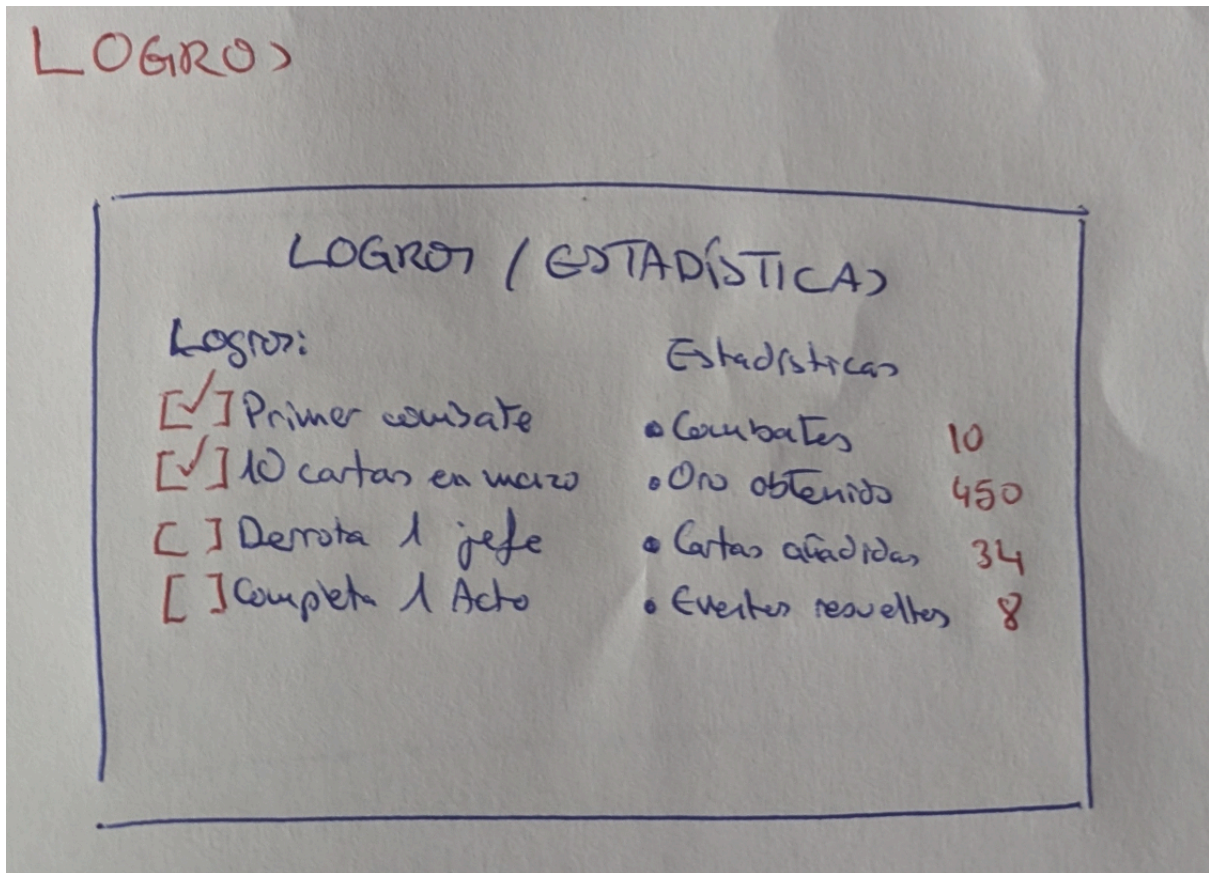
Botón:

- Volver

2.2 Interacción Opciones

- Modificar volumen
- Volver → Pantalla de Menú principal

1.3 Logros



Títulos

Binarios de logros

Estadísticas numéricas

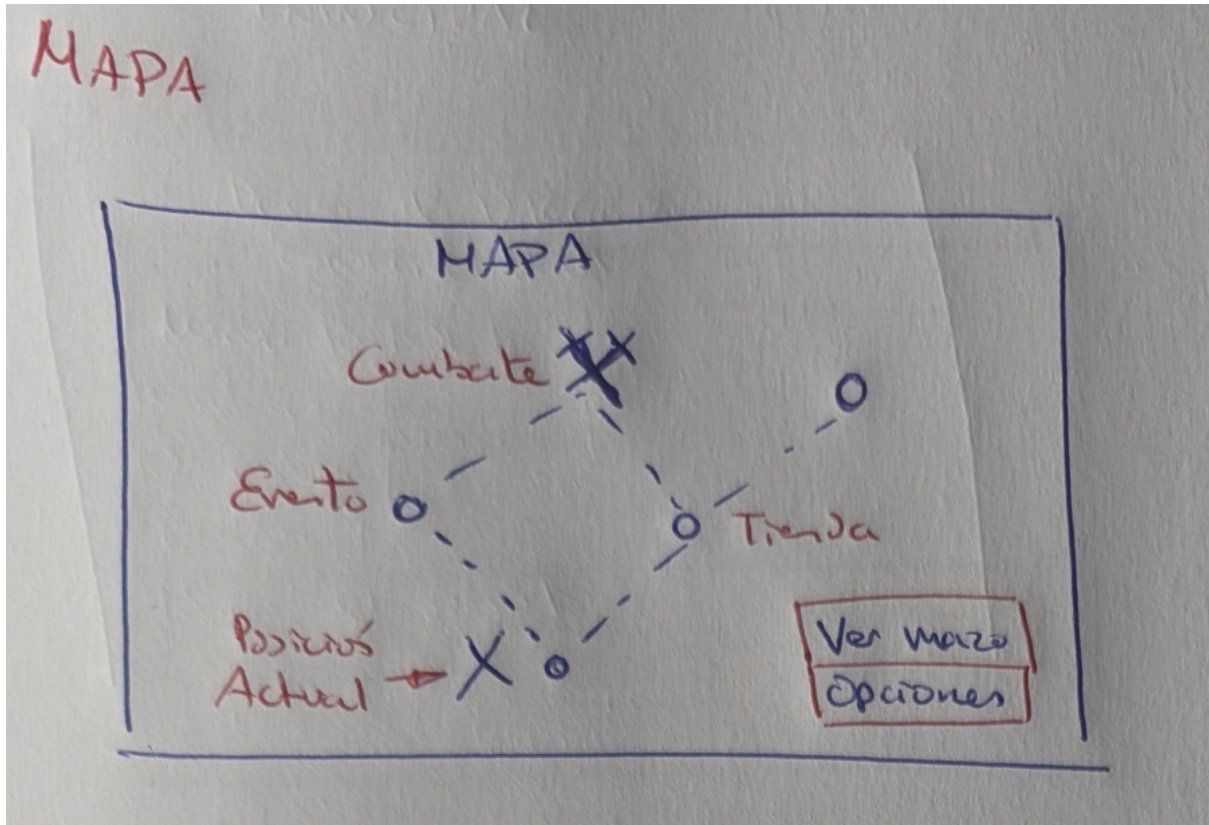
Botón:

- Volver (Falta en el boceto)

2.3 Interacción Logros

- Visualizar logros y estadísticas
- Volver → Pantalla de Menú principal

1.4 Mapa



Título

Posición Actual jugador

Nodos:

- Evento
- Tienda
- Combate

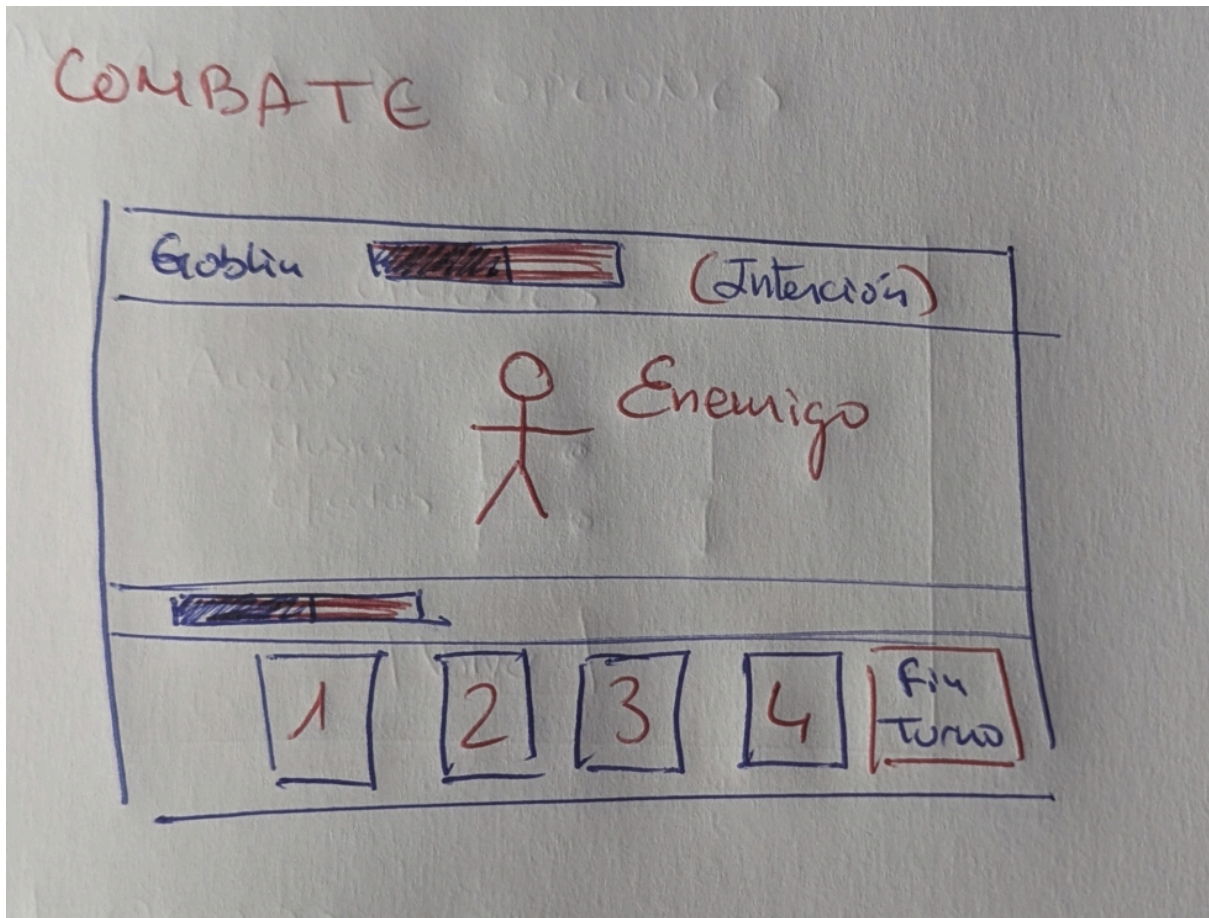
Botones:

- Ver mazo
- Opciones

2.4 Interacción Mapa

- Seleccionar un nodo de combate → Combate
- Seleccionar un nodo de evento → Evento
- Seleccionar un nodo de tienda → Tienda
- Pulsar Ver Mazo → Pantalla Mazo
- Abrir Opciones → Pantalla Opciones

1.5 Combate

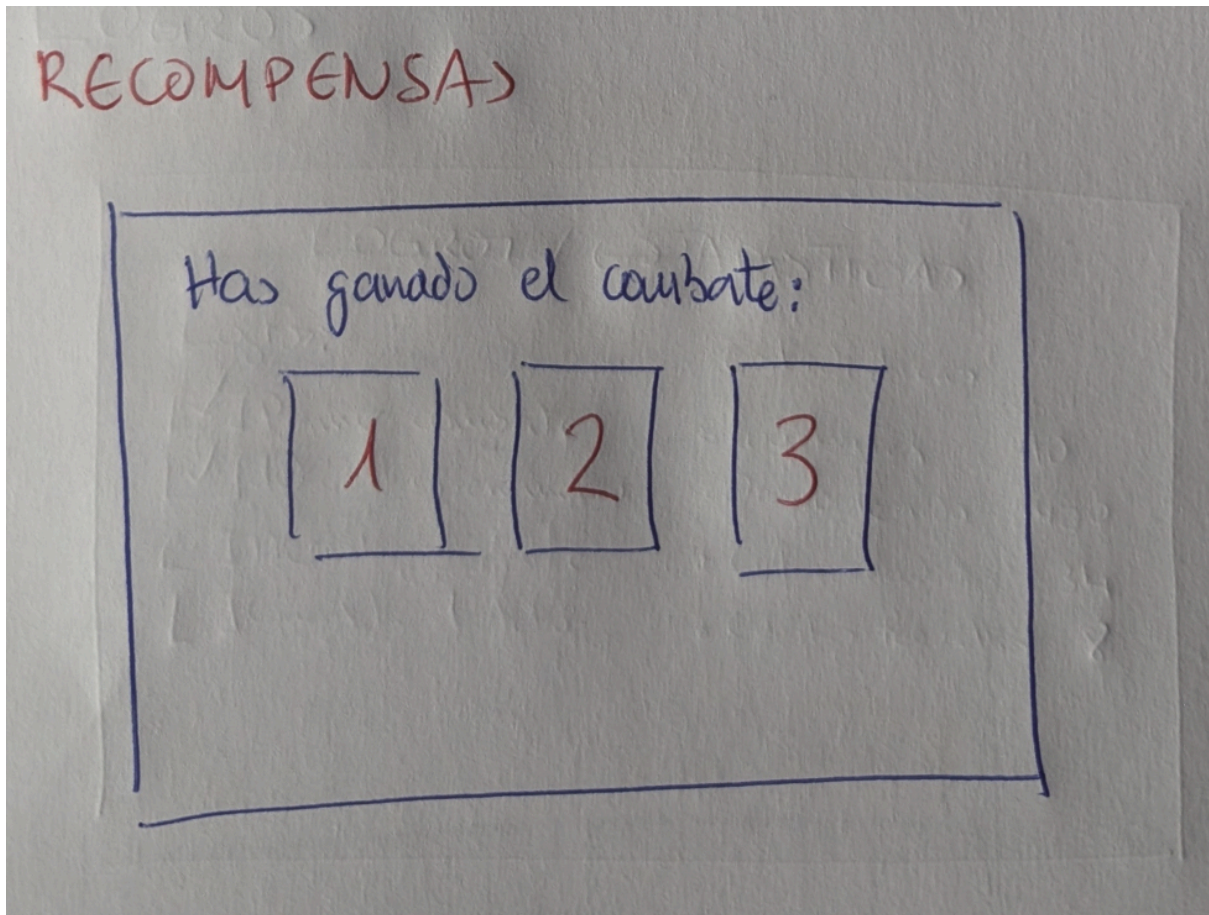


Nombre enemigo
Vida enemigo
Intención enemigo
Imagen enemigo
Vida jugador
Cartas
Botón:
- Fin de turno

2.5 Interacción Combate

- Juega turnos hasta ganar o perder.
- Si pierde → Fin de partida (no incluye boceto si no lo necesitas).
- Si gana → Pantalla de Recompensas

1.6 Recompensa



Selección de 3 opciones de carta

2.6 Interacción Recompensas

- Al seleccionar una carta → Pantalla mapa

1.7 Evento

A hand-drawn sketch of an event form on a piece of paper. At the top, the word 'EVENTO' is written in red capital letters. Below it is a rectangular box with a blue border. Inside the box, the word 'Título' is written in blue ink at the top. Below that, the word 'Texto' is written in blue ink. At the bottom of the box, there are three red-outlined rectangular boxes, each containing the text 'Opción 1', 'Opción 2', and 'Opción 3' respectively, separated by vertical lines.

Título

Texto narrativo

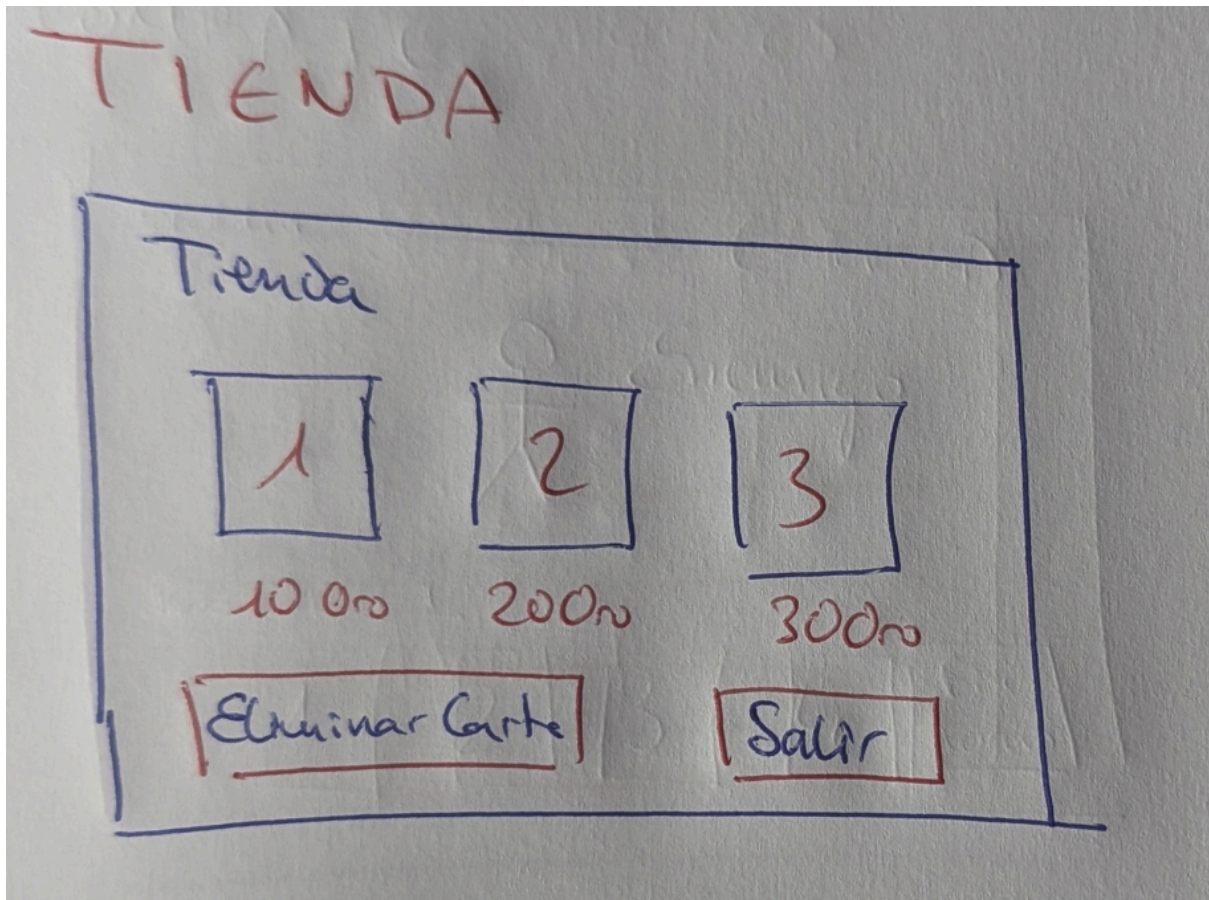
Botones:

- Opción 1
- Opción 2
- Opción 3

2.7 Interacción Evento

- Lee la narrativa
- Selecciona una opción disponible
- Se aplican las consecuencias
- Regresa al mapa

1.8 Tienda



Título

3 opciones de compra

Precios de las cartas

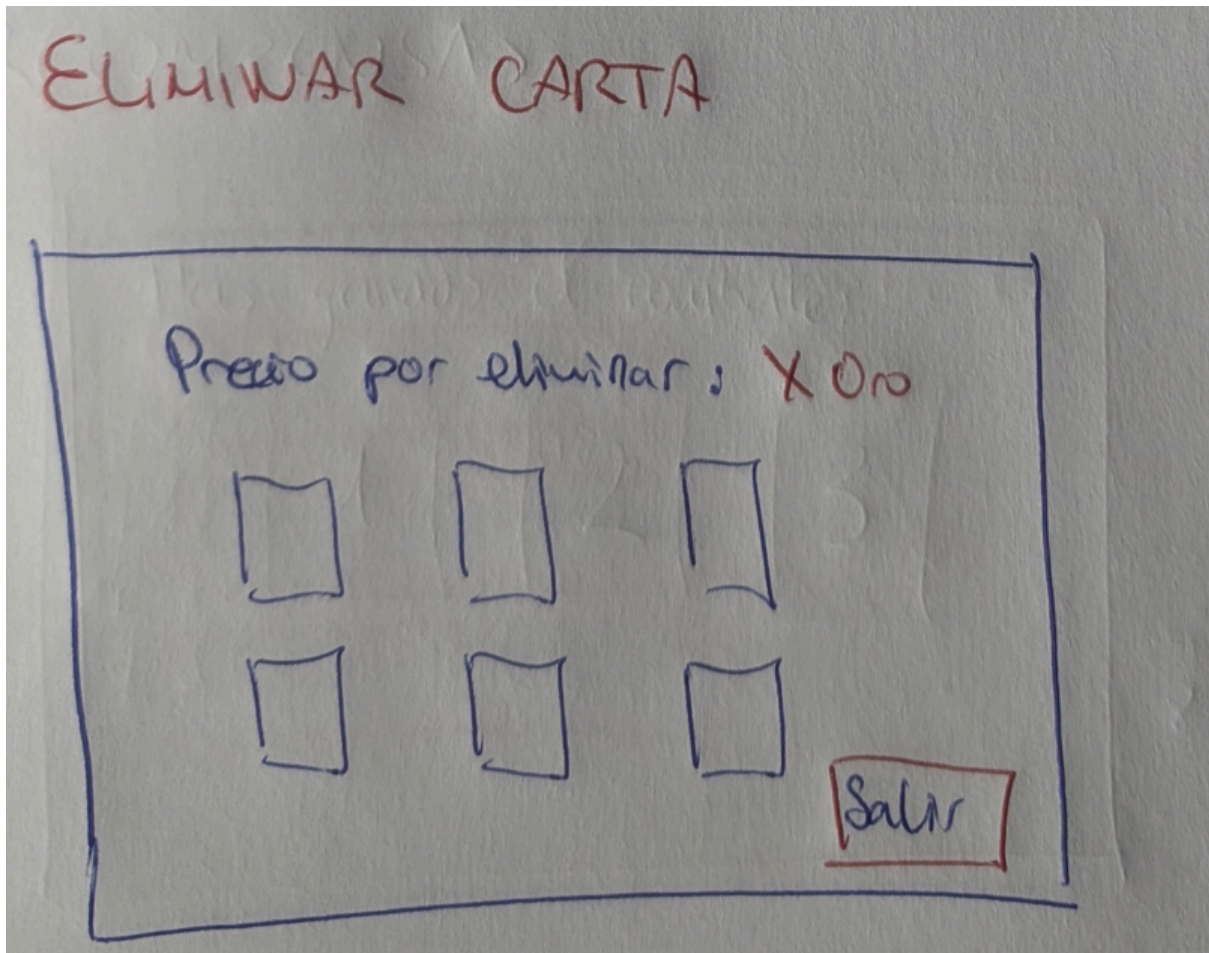
Botones:

- Eliminar carta
- Salir

2.8 Interacción Tienda

- Comprar cartas
- Acceder a Eliminar carta
- Salir → volver al mapa

1.9 Eliminar carta



Precio por eliminar carta

Cartas a eliminar

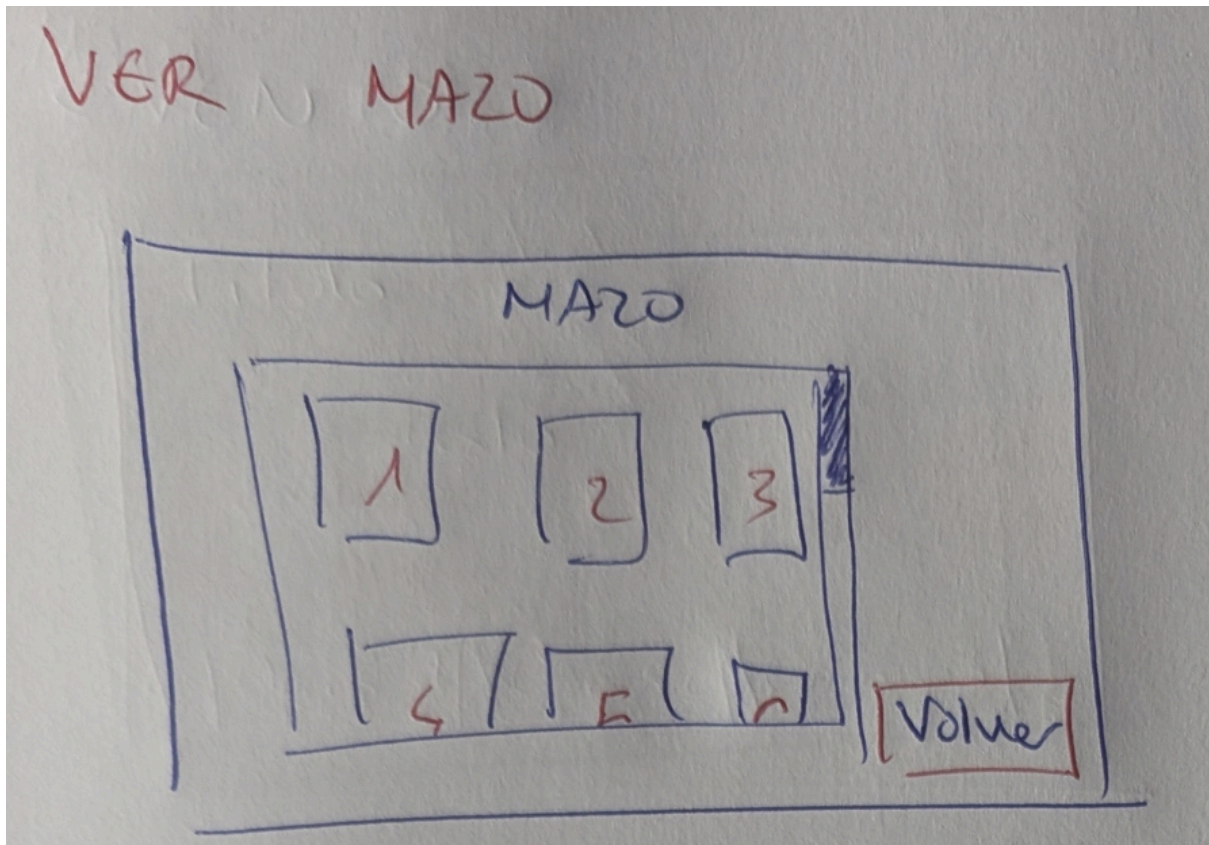
Botón:

- Salir

2.9 Interacción Eliminar carta

- Elimina carta si paga
- Salir → Volver al mapa

1.10 Mazo



Título

Container de cartas

Slider de container

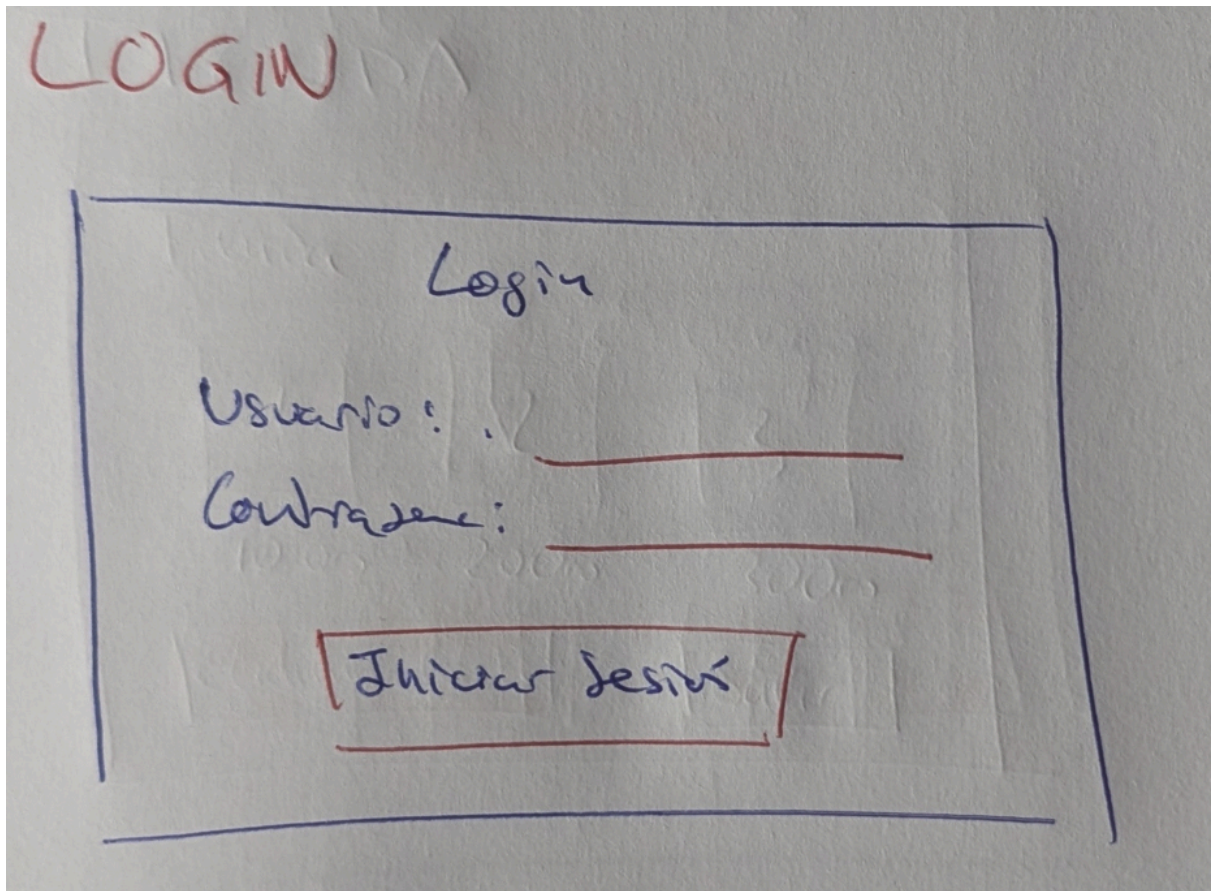
Botón:

- Volver

2.10 Interacción Mazo

- Visualizar cartas del mazo
- Volver → Pantalla mapa

1.11 Login



Título

Usuario y espacio escritura.

Contraseña y espacio escritura

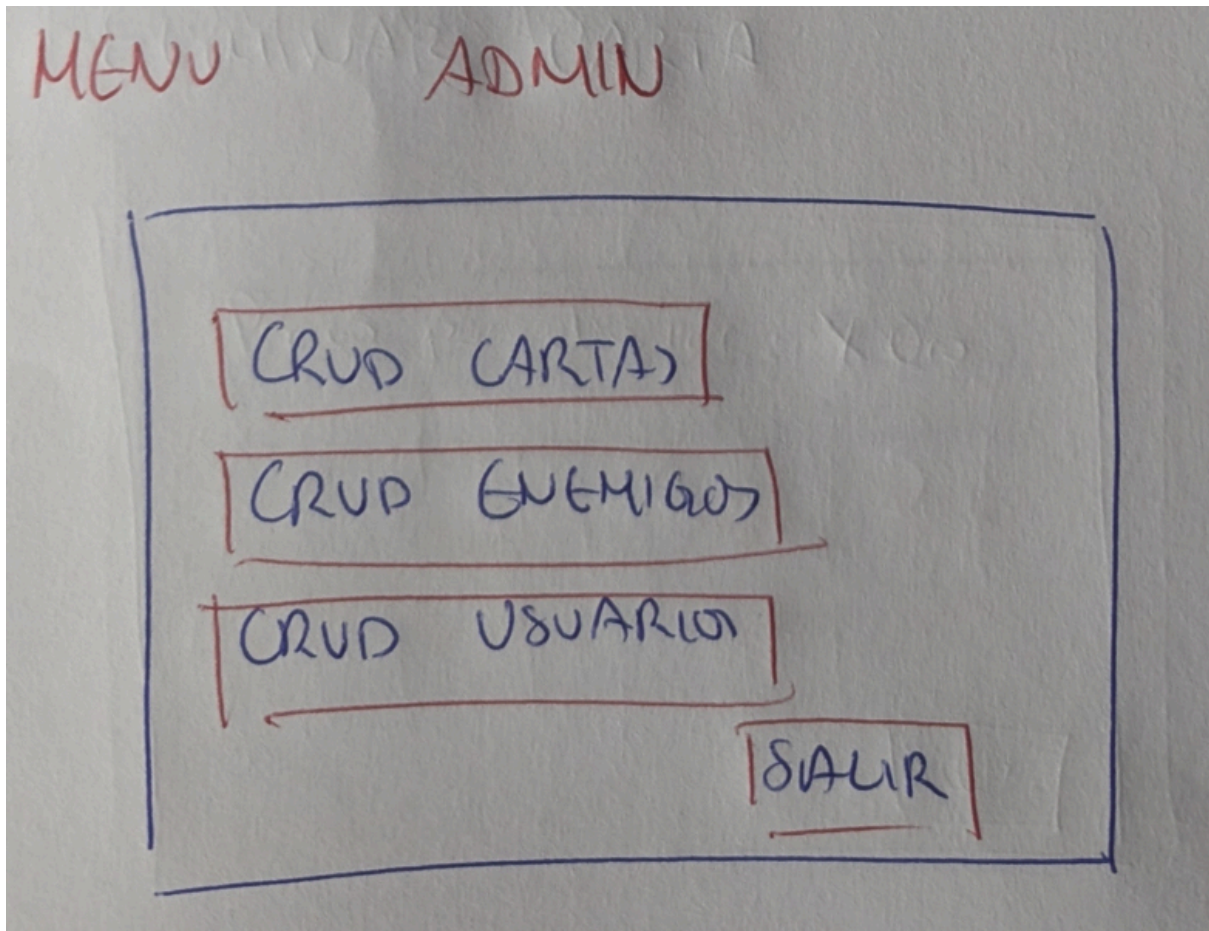
Botón:

- Iniciar sesión

2.11 Interacción login

- Escribir usuario y contraseña
- Si son correctos acceder al menu admin

1.12 Menú Admin



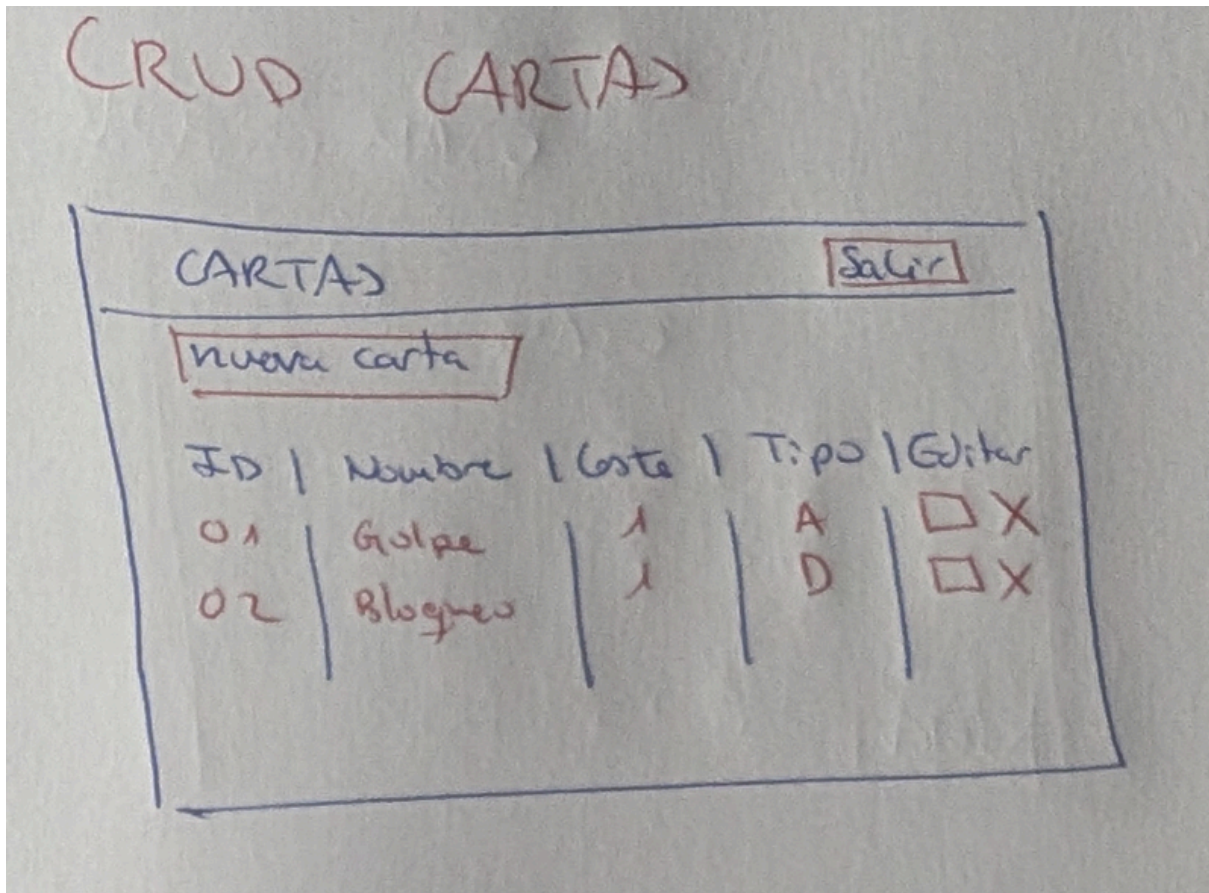
Botones:

- CRUD Cartas
- CRUD Enemigos
- CRUD Usuarios
- Salir

2.12 Interacción Menú Admin

- CRUD Cartas → Pantalla de CRUD Cartas
- CRUD Enemigos → Pantalla de CRUD Enemigos
- CRUD Usuarios → Pantalla de CRUD Usuarios
- Salir → Cierra la app

1.13 CRUD Cartas



Título

Botones:

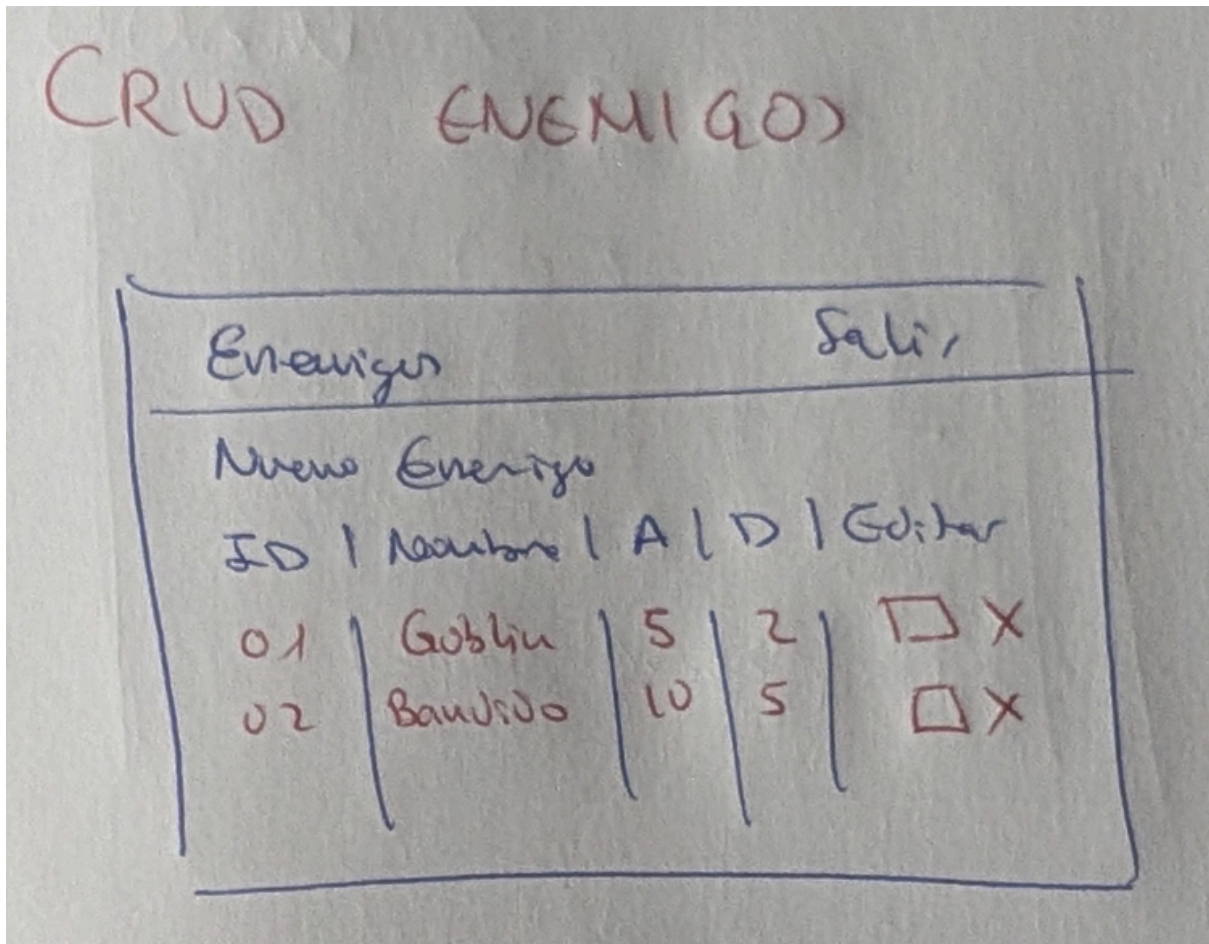
- Salir
- Nueva Carta
- Editar (de cada carta)
- Eliminar (de cada carta)

Tabla de cartas

2.13 Interacción CRUD Cartas

- Crear cartas
- Editar cartas
- Visualizar cartas
- Eliminar cartas
- Salir

1.14 CRUD Enemigos



Título

Botones:

- Salir
- Nuevo Enemigo
- Editar (de cada enemigo)
- Eliminar (de cada enemigo)

Tabla de enemigos

2.14 Interacción CRUD Enemigos

- Crear enemigos
- Editar enemigos
- Visualizar enemigos
- Eliminar enemigos
- Salir

1.15 CRUD Usuarios

CRUD USUARIO

Usuarios			Salir
ID	Nombre	Rol	Cambiar
01	Admin	Admin	☐ X
02	Andrés	Jugador	☐ X

Título

Botones:

- Salir
- Cambiar (para cada usuario)
- Eliminar (para cada usuario)

2.15 Interacción CRUD Usuarios

- Cambiar el rol de los usuarios (minimo un admin)
- Eliminar usuarios
- Visualizar usuarios

2. Diseño de la aplicación

2.1. Estudio de viabilidad

1. Viabilidad Operacional

1.1 Estructura general del trabajo

El desarrollo se planifica bajo una metodología Scrum, con sprints de una semana de duración. Cada sprint se plantea como un incremento funcional verificable que aporta valor al producto final.

El alcance temporal total es de 2 meses, comprendiendo 8 sprints, con una dedicación media de 20 a 30 horas semanales, lo que supone aproximadamente 200 horas efectivas de trabajo.

1.2 Fases y tareas principales

Sprint 1

Análisis y planificación: Estudios de mercado, Ayudas y subvenciones, Especificación de requisitos y demás tareas previas.

30 horas

Sprint 2

Diseño de clases, interfaces, biblioteca de cartas y enemigos.

25 horas.

Sprint 3-4

Desarrollo de las estructuras base y motor de combate. Sistema de cartas, gestión de turnos, enemigos, tipos de ataque.

50 horas

Sprint 5-6

Persistencia de datos y progresión del jugador. Integración de SQLite, desbloques, guardado de experiencia y estadísticas.

40 horas

Sprint 7

Integración de recursos gráficos y sonoros. Crear sprites, efectos de sonido, etc.

25 horas

Sprint 8

Pruebas y documentación. Pruebas funcionales, testeo, documentación técnica y manuales.

30 horas

2. Viabilidad de recursos

2.1 Recursos humanos

El proyecto es unipersonal, desempeñando el desarrollador todos los roles principales:

Director del proyecto

Planificación global, control de versiones, seguimiento de sprints.

Diseñador de juego

Diseño de mecánicas, balance de cartas, progresión del jugador.

Programador principal

Implementación de la lógica del juego, interfaz y base de datos.

Artista 2D

Creación de recursos gráficos básicos (iconos, fondos, interfaz).

Diseñador de sonido

Creación y edición de efectos y música.

Tester y documentador

Realización de pruebas, registro de incidencias, redacción de documentación técnica y manuales.

Dedicación estimada: entre 20 y 30 horas semanales: programación 60%, diseño 20% y documentación 20%.

2.2 Recursos tecnológicos

Motor de desarrollo	Godot Engine 4.5.1	Gratuito
Base de datos	SQLite	Gratuito
Control de versiones	GitHub	Gratuito
Edición gráfica	Krita	Gratuito
Edición de sonido	Cakewalk	Gratuito
Planificación	Github projects	Gratuito
Sistema operativo	Windows 11	Licencia propia
Internet	-	20€/mes
Ordenador	Parcial	200€
Electricidad	-	25€/mes
Distribución	Steam / Google Play	100€ tasas

3. Viabilidad financiera

3.1 Costes de personal

Concepto	Horas	Tarifa	Coste
Programación	120 h	15€/h	1.800€
Arte y sonido	40 h	12€/h	480€
Documentación y pruebas	40 h	12€/h	480€
Subtotal	200 h	–	2.760€

3.2 Costes de licencia

La mayoría de los recursos tienen licencias gratuitas por lo que solo pagaremos por la cuota de publicación única de Steam Direct y la cuota de alta de desarrollador de Google Play Console:

Steam Direct: 100€

Google Play Console: 25€

Subtotal: 125€

3.3 Costes de infraestructura

PC personal: Amortización parcial = 200€

Electricidad: 0,25€/h * 200h = 50€

Internet: 20€/mes 2 meses = 40€

Subtotal: 290€

3.4 Inversión total estimada

Costes de personal 2760€

Costes de licencia 125€

Costes de infraestructura 290€

Total: 3.175€

3.5 Viabilidad económica

El uso de herramientas libres y multiplataforma (Godot, SQLite, Krita, Cakewalk) reduce drásticamente los costes de producción.

El gasto principal recae en el valor del tiempo invertido, sin necesidad de inversión real significativa.

El coste efectivo real del proyecto se limita prácticamente a las tasas de publicación y consumo lo que lo hace totalmente viable dentro del marco académico puesto que en realidad el coste del personal es 0.

2.2. Planificación del proyecto

1. Objetivos y alcance

1.1 Objetivos

Objetivos generales

1. Desarrollar un videojuego tipo deckbuilder roguelike para PC, implementado en Godot 4.5.1 con GDScript y base de datos SQLite para el almacenamiento persistente y desbloques.
2. Aplicar metodologías ágiles (Scrum) en el desarrollo, documentación y control de versiones del proyecto, asegurando un flujo de trabajo iterativo, verificable y medible.
3. Consolidar los conocimientos adquiridos en el ciclo formativo DAM, integrando programación, diseño de base de datos, arquitectura de software y gestión de proyectos.

Objetivos específicos

1. Diseñar y documentar los requisitos funcionales, diagramas UML, base de datos relacional y estructura de clases.
2. Implementar los módulos principales del juego: menú principal, mapa de nodos, sistema de combate, tienda, eventos y biblioteca de cartas.
3. Desarrollar un sistema de progreso y guardado mediante SQLite, con registro de partidas, desbloqueo de cartas y estadísticas de jugador.
4. Crear una interfaz visual optimizada para diferentes resoluciones de PC, basada en los diseños definidos en la tarea 7.
5. Ejecutar un plan de pruebas para verificar el cumplimiento de los requisitos funcionales y no funcionales.
6. Elaborar toda la documentación técnica, manual de usuario y presentación final de defensa.

1.2 Alcance del Proyecto

Elementos incluidos

- Plataforma: versión funcional para PC (Windows).
- Modo de juego (single player)
- Contenido:
 - Tres mapas con enemigos progresivos y jefe final.
 - Combate por turnos con cartas de ataque, defensa y habilidad.
 - Sistema de tienda y eventos aleatorios.
 - Sistema de experiencia y desbloqueo de nuevas cartas.
 - Persistencia de datos y estadísticas.

Elementos fuera del alcance

- Versión Android: Considerando el tiempo previsto para la realización del proyecto, la versión Android se incluirá si no aparecen interferencias en el desarrollo de la aplicación.
- Funciones multijugador o en línea: Al tratarse de un juego singleplayer no hay necesidad de una función multijugador más allá de estadísticas o incluso un sistema de puntuación.

- Integración con plataformas externas (Steam, Google Play): Al tratarse de un desarrollo para Proyecto fin de Grado estimo innecesario la compra de licencias de publicación.
- Localización multilingüe: Considerando el tiempo previsto para la realización del proyecto, la versión multilingüe se incluirá si no aparecen interferencias en el desarrollo de la aplicación.
- Módulos de monetización o microtransacciones: Se trata de un juego indie premium lo que consisten en un solo pago inicial, como no lo vamos a integrar en plataformas externas no utilizaremos ningún módulo de monetización.

2. Ciclo de vida

El ciclo de vida Scrum se adapta perfectamente al desarrollo individual de un videojuego indie premium como Deck & Dagger: Goblin Hunt, ya que permite:

- Dividir el trabajo en Sprints semanales con entregables verificables (por ejemplo, combate, tienda, eventos, mapa).
- Detectar incidencias tempranas en la codificación y diseño, facilitando la mejora continua.
- Mantener una planificación flexible, adaptando los objetivos de cada iteración a los resultados obtenidos.
- Documentar el proceso mediante tableros Kanban.

Fases:

Inicio

Definición del proyecto, análisis de mercado y requisitos. Tareas 0-6 (Propuesta, análisis sectorial, DAFO, requisitos).

Diseño

Diseño visual, diagramas, estructura de base de datos. Tareas 7-12 (Bocetos, E/R, Relacional, Clases, Interfaces).

Planificación Scrum

Definición de Sprints, backlog y tablero Kanban. Tarea 15 (Planificación Proyecto - Scrum).

Desarrollo

Implementación funcional por módulos del juego. Sprints 1-8.

Pruebas y control de calidad

Planificación y ejecución de pruebas unitarias e integración. Tarea 14 (plan de pruebas)

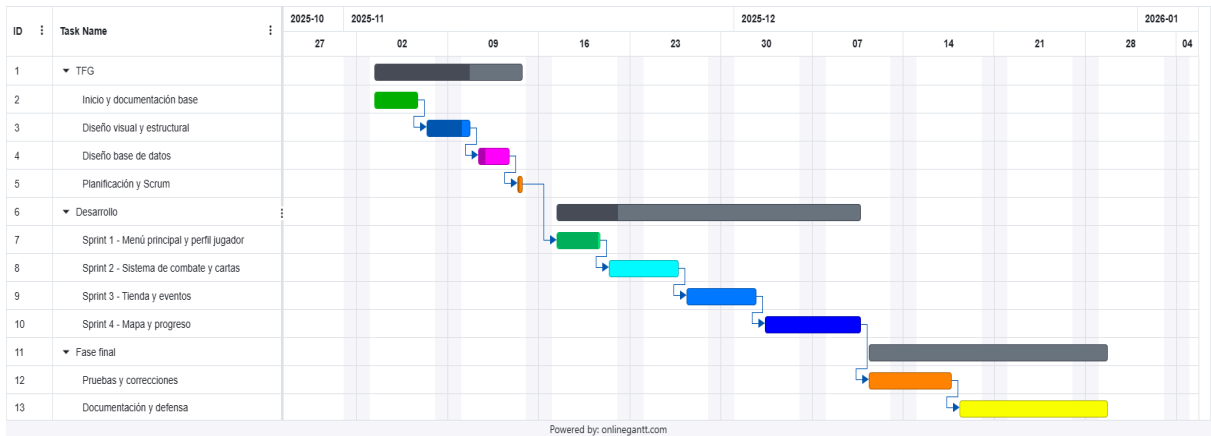
Seguimiento

Registro de incidencias y acciones correctivas. Tarea 16 (Seguimiento Proyecto - Scrum).

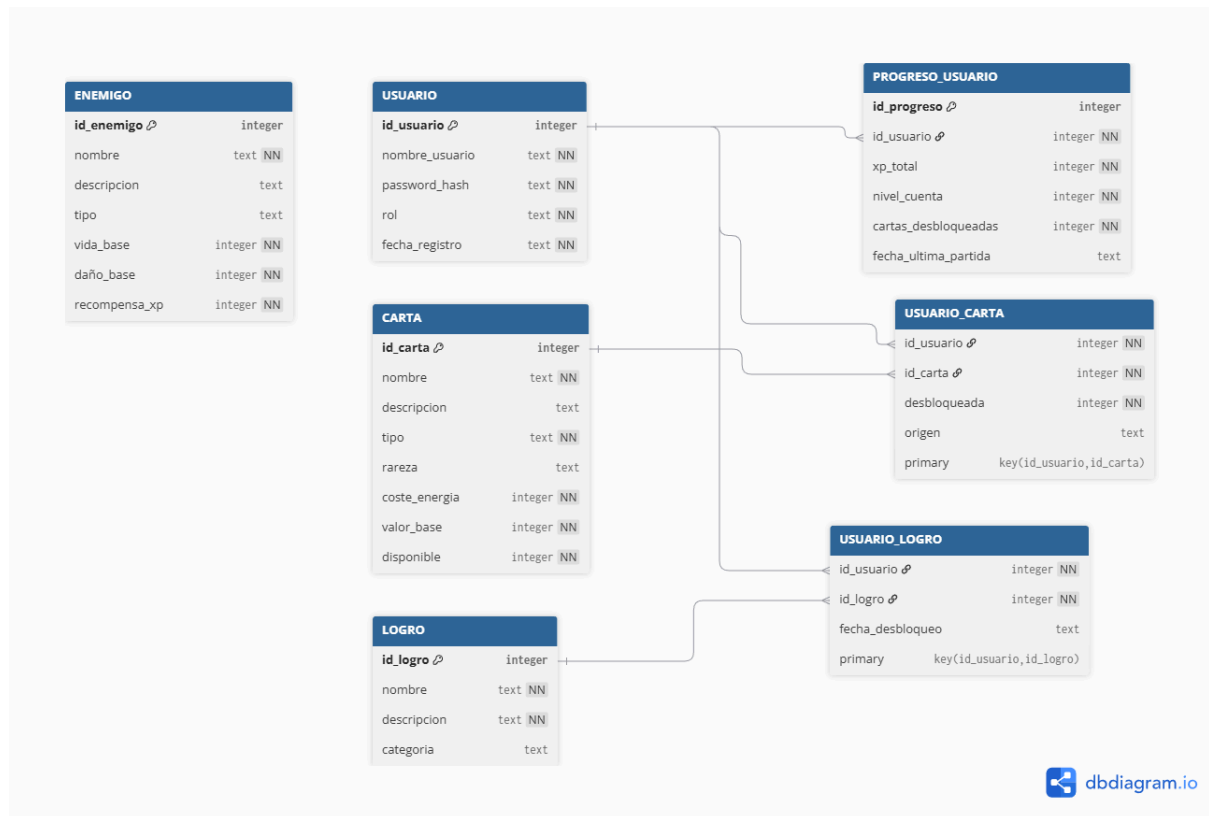
Documentación y defensa

Elaboración del manual técnico, presentación y defensa. Tareas 17-20.

3. Diagrama de Gantt



2.3. Diagrama Entidad Relación



2.4. Esquema Relacional

1. Tabla USUARIO

Descripción:

Almacena todos los usuarios del sistema, tanto jugadores como administradores.

Un registro representa un usuario del juego.

El rol determina si puede acceder al modo administrador.

Atributos:

id_usuario : INTEGER, NOT NULL

nombre_usuario : TEXT, NOT NULL

password_hash : TEXT, NOT NULL

rol : TEXT, NOT NULL

fecha_registro : TEXT, NOT NULL

Clave primaria:

PK(id_usuario)

Restricciones adicionales:

nombre_usuario es UNIQUE.

rol solo puede tomar valores lógicos: {'PLAYER', 'ADMIN'}

2. Tabla PROGRESO_USUARIO

Descripción:

Almacena el progreso global de la cuenta de cada usuario (metajuego).

Cada usuario tiene como máximo un registro de progreso.

Se guarda la XP global, el nivel de la cuenta y un contador de cartas desbloqueadas.

Atributos:

id_progreso : INTEGER, NOT NULL

id_usuario : INTEGER, NOT NULL

xp_total : INTEGER, NOT NULL

nivel_cuenta : INTEGER, NOT NULL

cartas_desbloqueadas : INTEGER, NOT NULL

fecha_ultima_partida : TEXT, NULL

Clave primaria:

PK(id_progreso)

Claves externas:

FK(id_usuario) → USUARIO(id_usuario)

Restricciones adicionales:

id_usuario es UNIQUE en esta tabla → relación 1:1 entre USUARIO y PROGRESO_USUARIO.

3. Tabla CARTA

Descripción:

Catálogo maestro de cartas disponibles en el juego.

Cada registro define una carta jugable (o desactivada si disponible = 0).

Este catálogo lo usará tanto el jugador como el modo admin.

Atributos:

id_carta : INTEGER, NOT NULL

nombre : TEXT, NOT NULL

descripcion : TEXT, NULL

tipo : TEXT, NOT NULL

rareza : TEXT, NULL

coste_energia : INTEGER, NOT NULL

valor_base : INTEGER, NOT NULL

disponible : INTEGER, NOT NULL

Clave primaria:

PK(id_carta)

Restricciones adicionales:

nombre es UNIQUE.

disponible es un valor lógico {0,1}.

tipo puede tener un dominio lógico (por ejemplo): {'ATAQUE', 'DEFENSA', 'HABILIDAD', 'MALDICION', ...}.

4. Tabla ENEMIGO

Descripción:

Catálogo de enemigos que aparecen en el prototipo.

Define las características básicas de los enemigos (vida, daño y XP que dan).

El modo admin puede crear, modificar y borrar enemigos del catálogo.

Atributos:

id_enemigo : INTEGER, NOT NULL
nombre : TEXT, NOT NULL
descripcion : TEXT, NULL
tipo : TEXT, NULL
vida_base : INTEGER, NOT NULL
daño_base : INTEGER, NOT NULL
recompensa_xp : INTEGER, NOT NULL

Clave primaria:

PK(id_enemigo)

Restricciones adicionales:

nombre es UNIQUE.

5. Tabla LOGRO

Descripción:

Catálogo de logros disponibles en el juego.
Cada registro representa un logro desbloqueable (por progreso, combate, colección, etc.).

Atributos:

id_logro : INTEGER, NOT NULL
nombre : TEXT, NOT NULL
descripcion : TEXT, NOT NULL
categoria : TEXT, NULL

Clave primaria:

PK(id_logro)

Restricciones adicionales:

nombre es UNIQUE.

6. Tabla USUARIO_CARTA

Descripción:

Tabla intermedia para la relación N:M entre USUARIO y CARTA.
Indica qué cartas tiene desbloqueadas cada usuario.
Origen permite saber si esa carta es inicial, la ha ganado por un logro, por progreso, etc.

Atributos:

id_usuario : INTEGER, NOT NULL
id_carta : INTEGER, NOT NULL
desbloqueada : INTEGER, NOT NULL
origen : TEXT, NULL

Clave primaria:

PK(id_usuario, id_carta) (clave primaria compuesta)

Claves externas:

FK(id_usuario) → USUARIO(id_usuario)
FK(id_carta) → CARTA(id_carta)

Restricciones adicionales:

desbloqueada es un valor lógico {0,1}.
La combinación (id_usuario, id_carta) es única por la PK, un usuario no puede tener la misma carta duplicada en esta tabla.

7. Tabla USUARIO_LOGRO

Descripción:

Tabla intermedia para la relación N:M entre USUARIO y LOGRO.
Registra qué logros ha conseguido cada usuario y cuándo los desbloqueó.

Atributos:

id_usuario : INTEGER, NOT NULL
id_logro : INTEGER, NOT NULL
fecha_desbloqueo : TEXT, NULL

Clave primaria:

PK(id_usuario, id_logro) (clave primaria compuesta)

Claves externas:

FK(id_usuario) → USUARIO(id_usuario)
FK(id_logro) → LOGRO(id_logro)

Restricciones adicionales:

La combinación (id_usuario, id_logro) es única por la PK, un mismo logro no puede registrarse dos veces para el mismo usuario.

Resumen de relaciones entre tablas

USUARIO – PROGRESO_USUARIO

Tipo: 1:1

Implementación: FK PROGRESO_USUARIO.id_usuario referencia a USUARIO.id_usuario y es UNIQUE.

Semántica: cada usuario tiene, como máximo, un registro de progreso global.

USUARIO – CARTA (a través de USUARIO_CARTA)

Tipo: N:M

Implementación: tabla intermedia USUARIO_CARTA con PK compuesta (id_usuario, id_carta) y FKs a USUARIO y CARTA.

Semántica: un usuario puede tener muchas cartas desbloqueadas, y una misma carta puede pertenecer a muchos usuarios.

USUARIO – LOGRO (a través de USUARIO_LOGRO)

Tipo: N:M

Implementación: tabla intermedia USUARIO_LOGRO con PK compuesta (id_usuario, id_logro) y FKs a USUARIO y LOGRO.

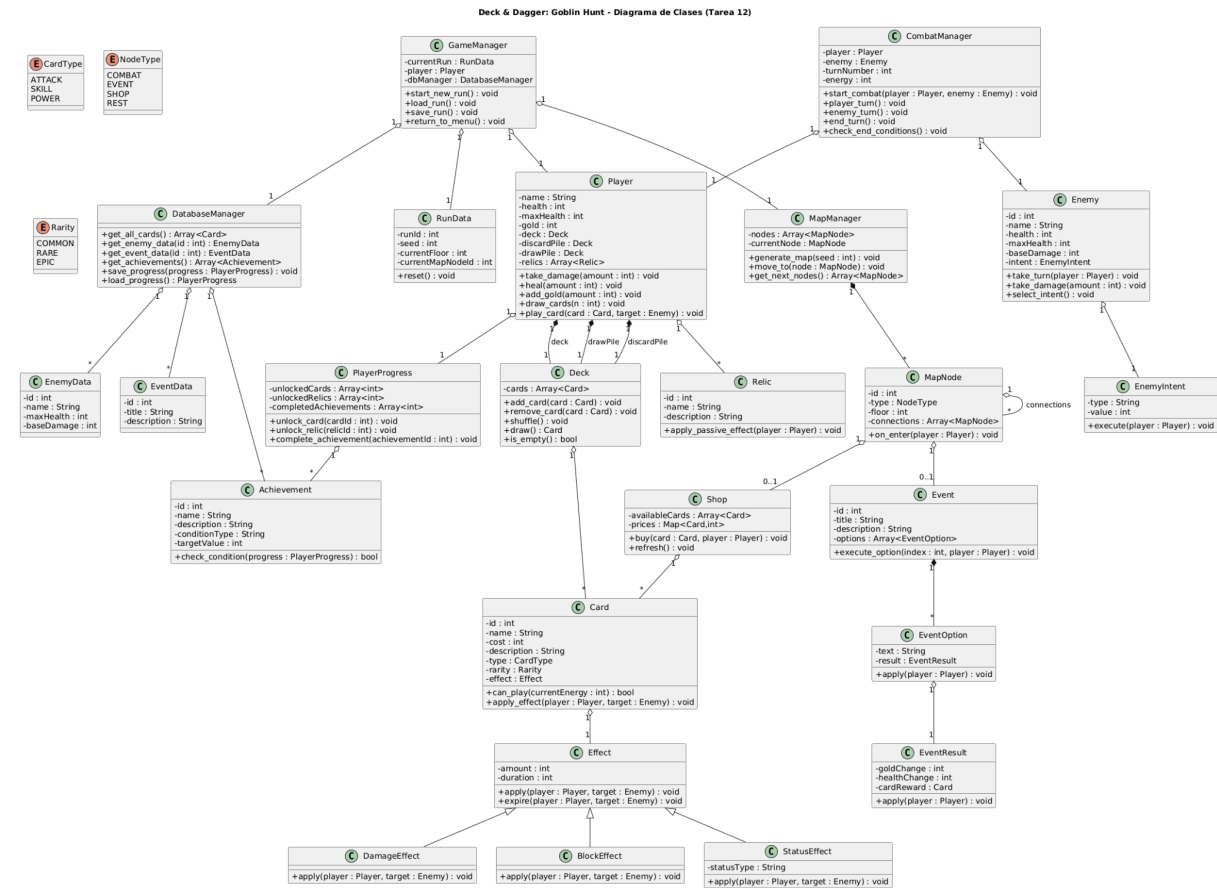
Semántica: un usuario puede desbloquear muchos logros, y un logro puede ser desbloqueado por muchos usuarios.

ENEMIGO

En este esquema relacional actúa como catálogo independiente.

Se relacionará con la lógica de combate a nivel de aplicación.

2.5. Diagrama de Clases



2.6. Diseño de interfaces

1. Sistema de diseño

En esta tarea se presenta el diseño de interfaces del proyecto Deck & Dagger: Goblin Hunt, un videojuego del género deckbuilder roguelike desarrollado con el motor Godot 4.5.1.

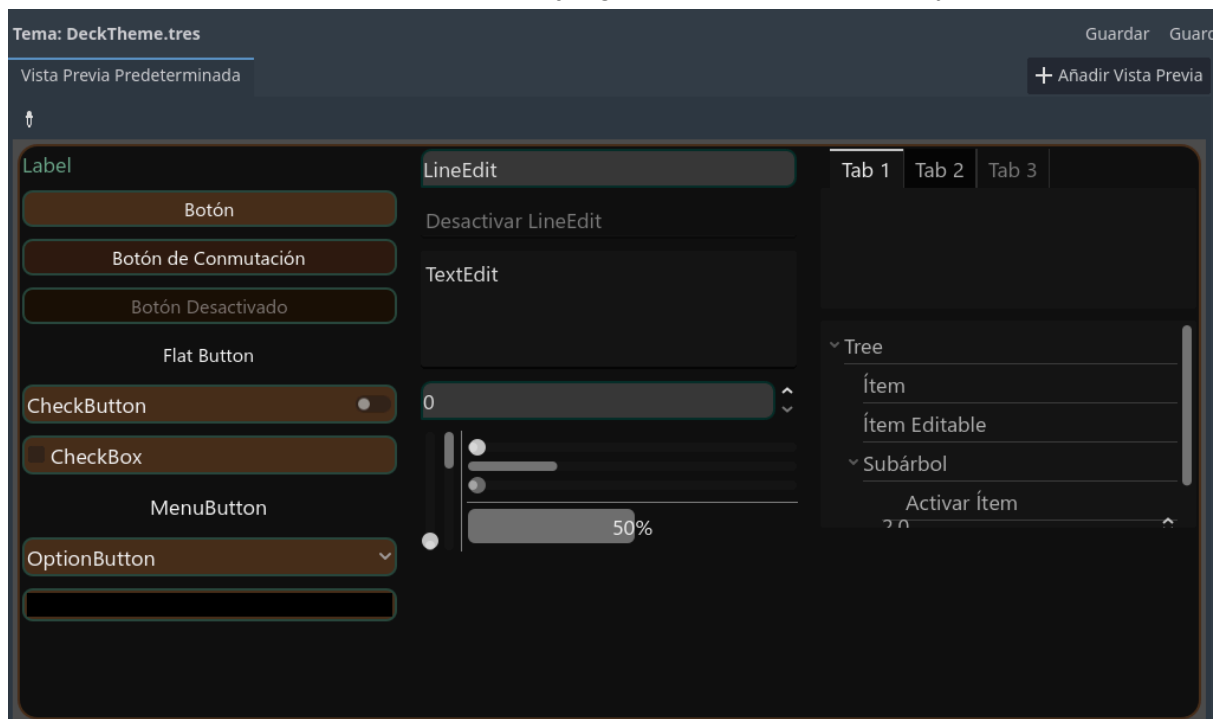
El objetivo de esta tarea es definir y describir las distintas pantallas del sistema, tanto para el jugador como para el administrador, asegurando una navegación coherente, una experiencia de usuario clara y un diseño visual consistente.

El diseño de las interfaces se ha basado en los siguientes principios:

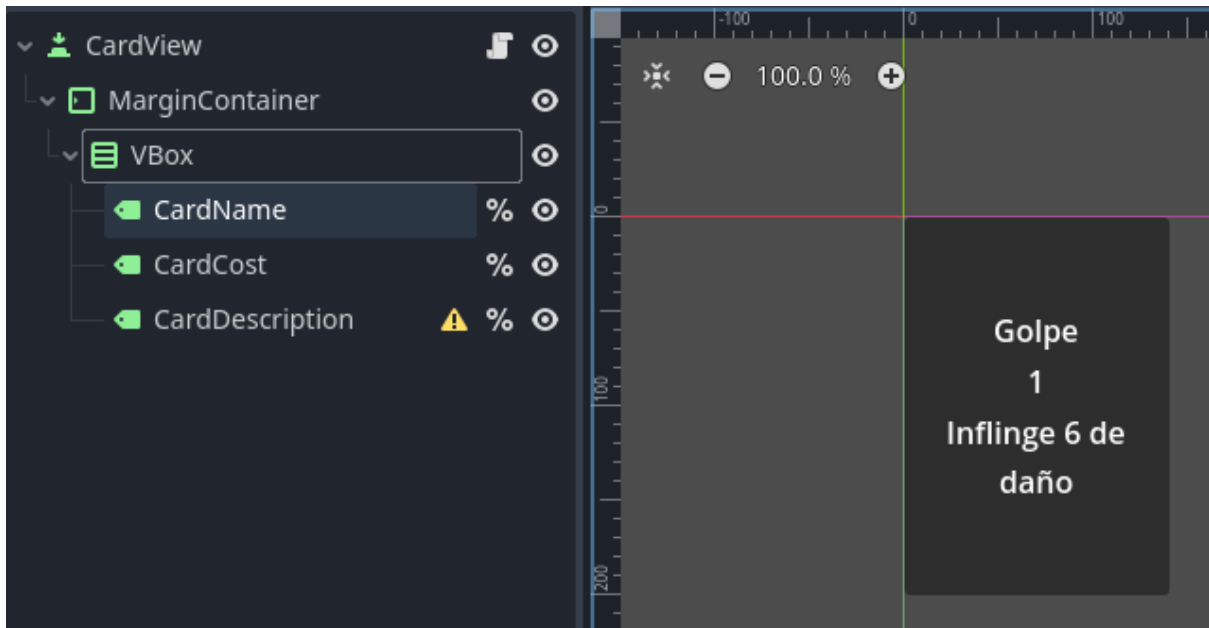
- Minimalismo visual, priorizando la funcionalidad sobre el detalle gráfico.
- Coherencia estética, utilizando una paleta de colores oscura inspirada en un entorno de bosque y poblado goblin.
- Separación de roles, diferenciando claramente las interfaces de jugador y de administrador.
- Usabilidad, con controles grandes, textos legibles y navegación intuitiva.
- Escalabilidad, permitiendo añadir lógica y persistencia de datos sin rediseñar las pantallas.

Todas las interfaces se han desarrollado íntegramente dentro de Godot, utilizando nodos Control y contenedores (VBoxContainer, HBoxContainer, GridContainer, etc.).

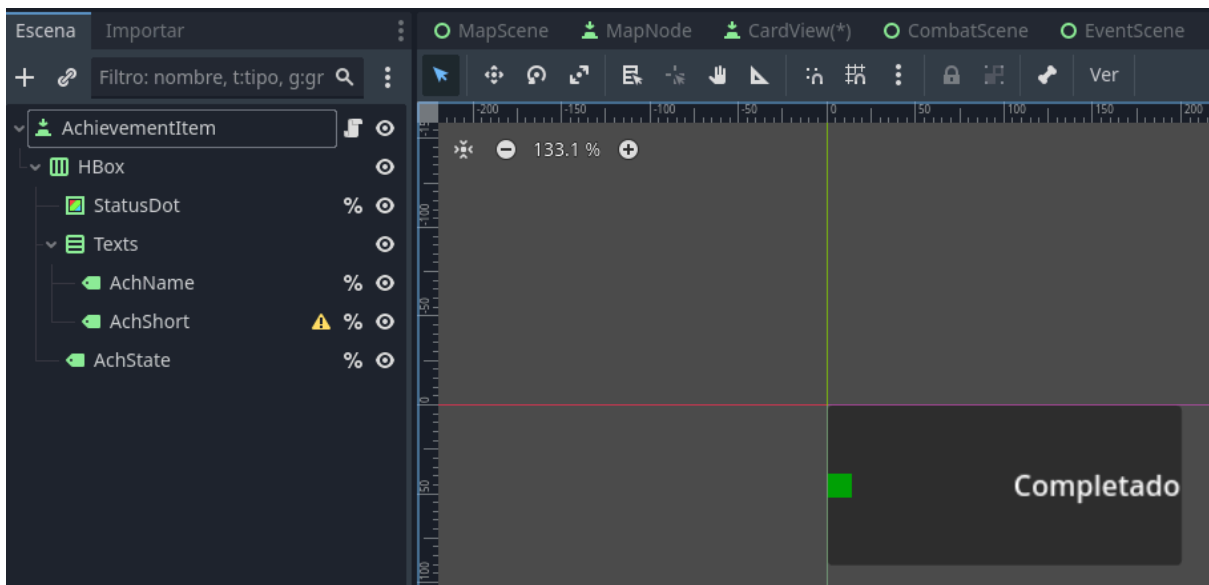
Se han creado un Tema base para todo el juego basándose en bosque y madera:



Y elementos reutilizables como las cartas CardView.tscn:



O logros AchievementItem.tscn:



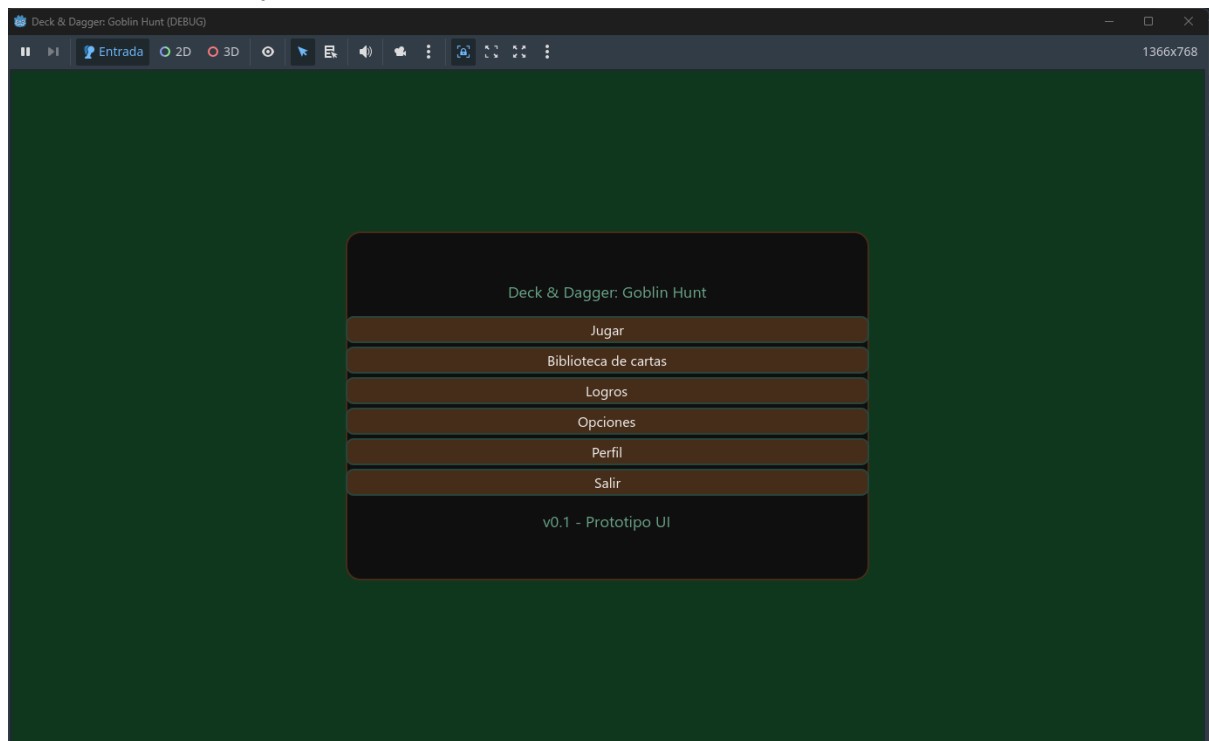
2. Interfaces del juego

2.1 Menu principal

El menú principal actúa como punto de entrada a la aplicación.
Desde él se puede acceder a:

Iniciar partida
Biblioteca de cartas
Logros
Opciones
Gestión de perfil

La interfaz es clara y directa, facilitando el acceso a las principales funcionalidades.

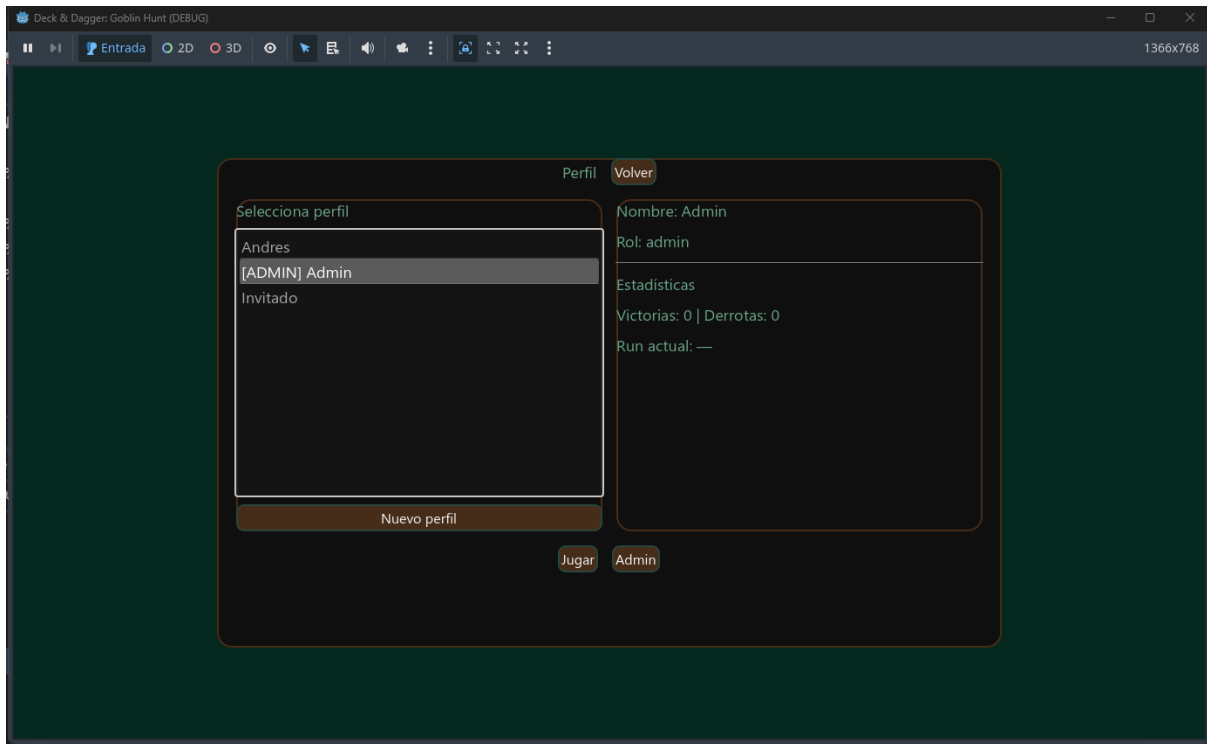


2.2 Perfil de jugador

La pantalla de perfil permite seleccionar el perfil activo antes de iniciar una partida.
Se muestra información básica como:

- Nombre del perfil
- Rol (jugador o administrador)
- Estadísticas generales
- Progreso de la run actual

Desde esta pantalla se puede iniciar una partida, entrar en modo admin/gestión o volver al menú principal.

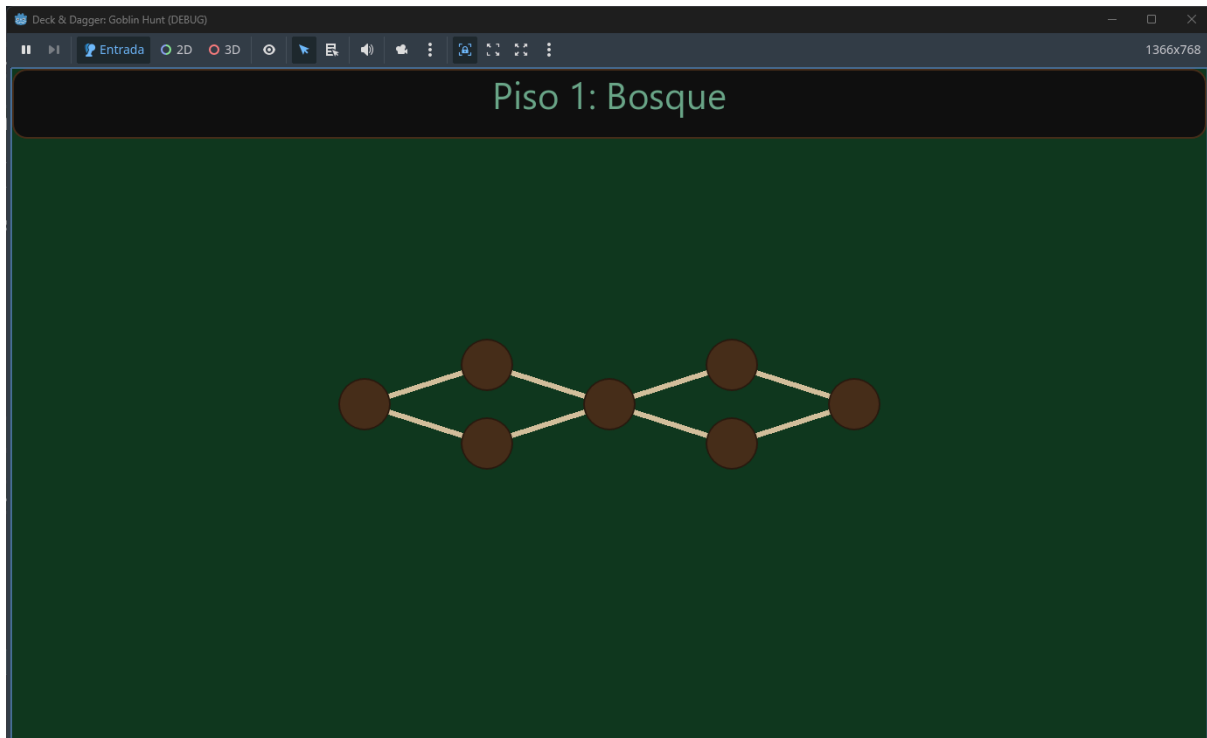


2.3 Mapa

El mapa representa el avance del jugador a través de la run, con nodos conectados que pueden corresponder a:

- Combates
- Eventos
- Tiendas

El jugador selecciona el siguiente nodo para continuar su progreso.

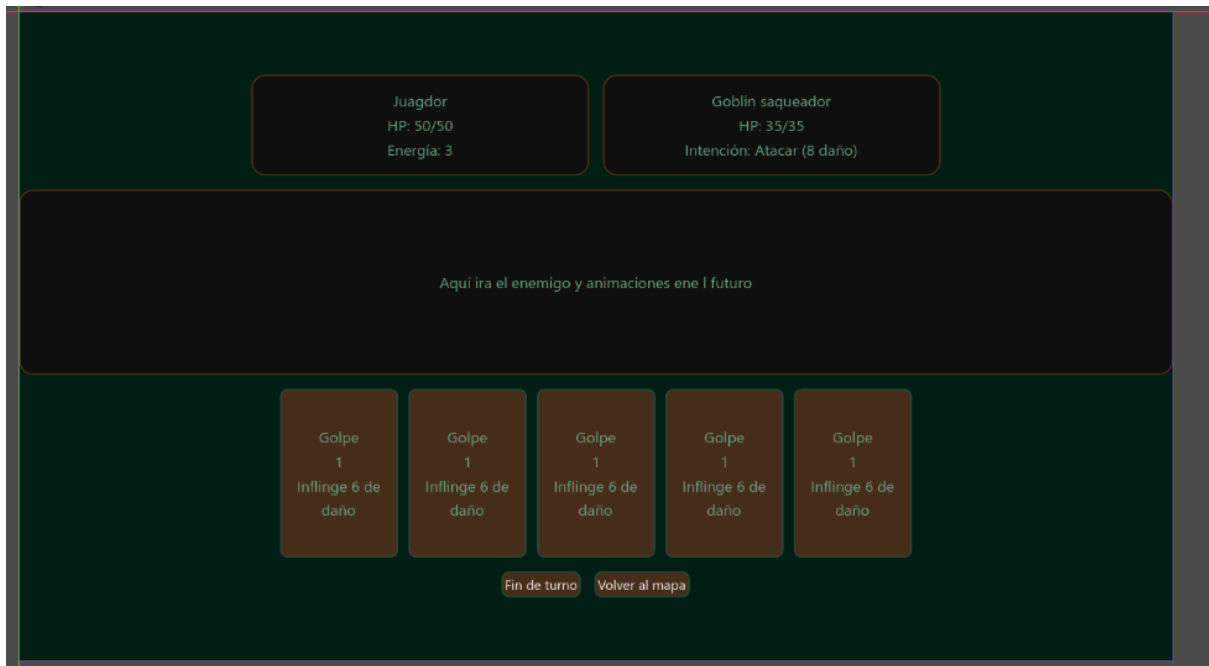


2.4 Combate

La interfaz de combate muestra:

- Información del jugador y del enemigo
- Mano de cartas
- Zona central de combate
- Acciones disponibles (fin de turno, etc.)

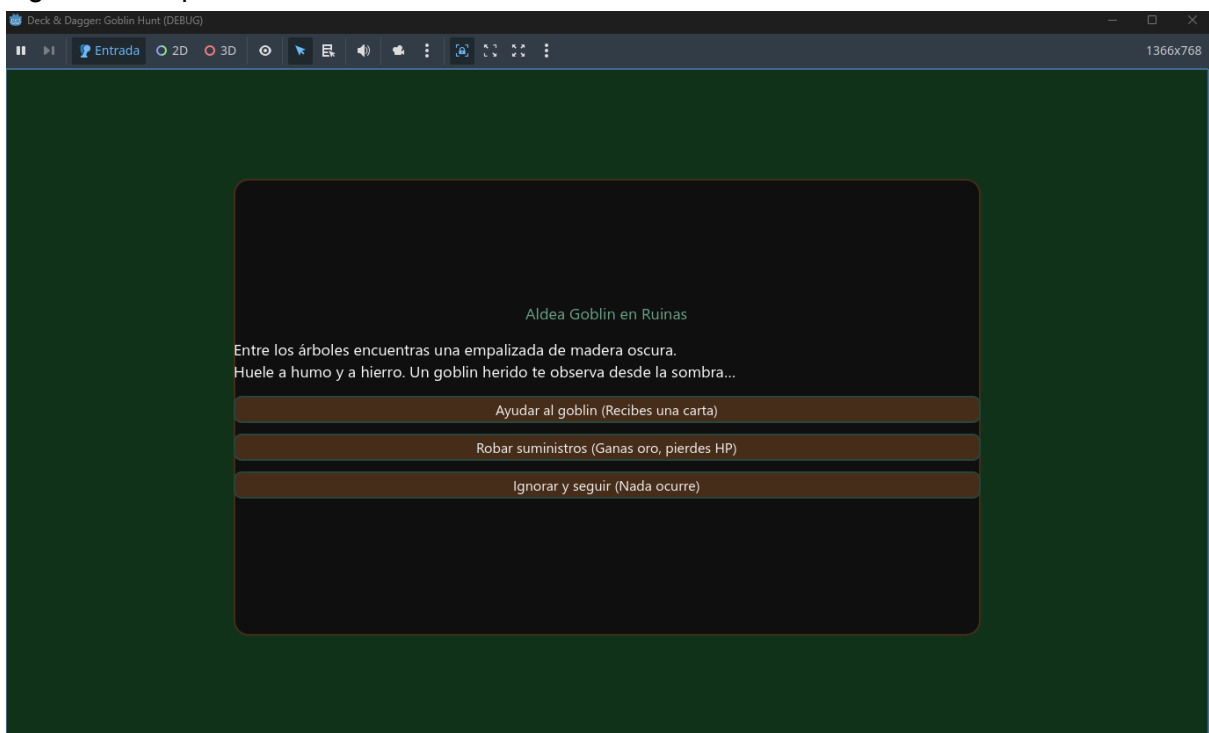
El diseño es minimalista y está preparado para incorporar animaciones y lógica avanzada en fases posteriores.



2.5 Eventos

Los eventos presentan situaciones narrativas con varias opciones posibles.

El jugador debe elegir una opción obligatoriamente, tras lo cual se muestra el resultado y se regresa al mapa.

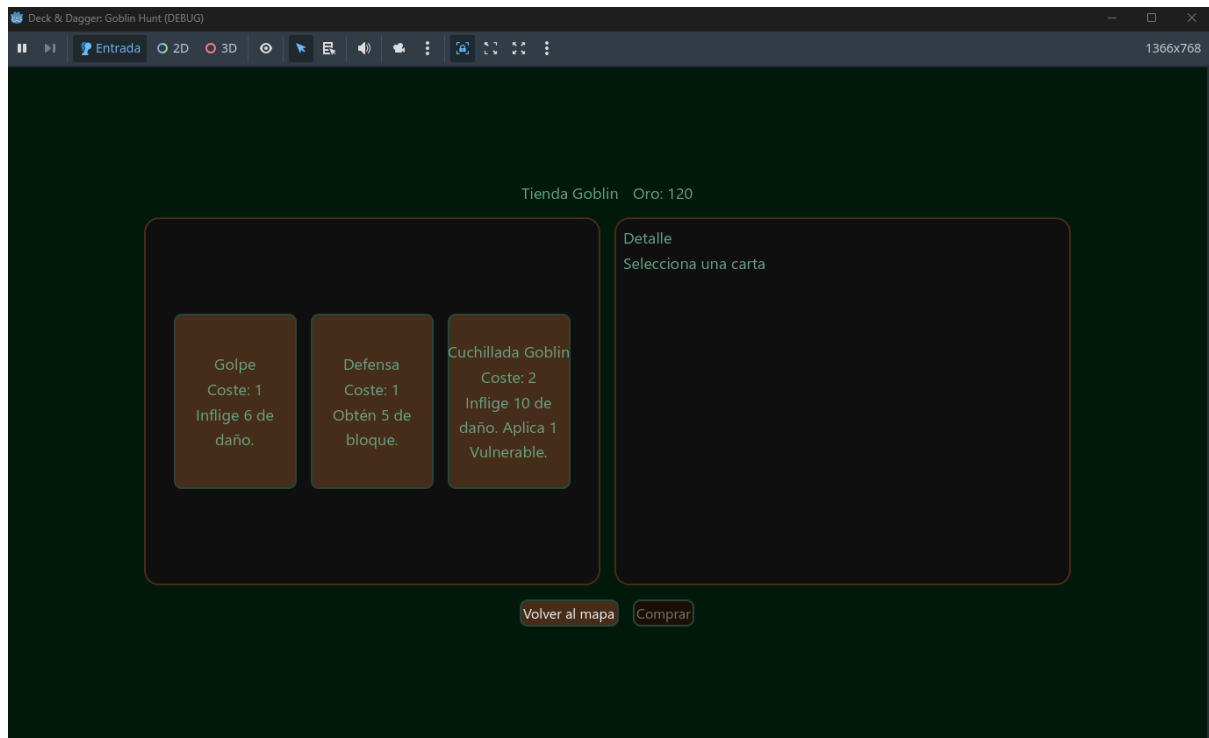


2.6 Tienda

La tienda permite adquirir cartas utilizando oro.

La interfaz se divide en:

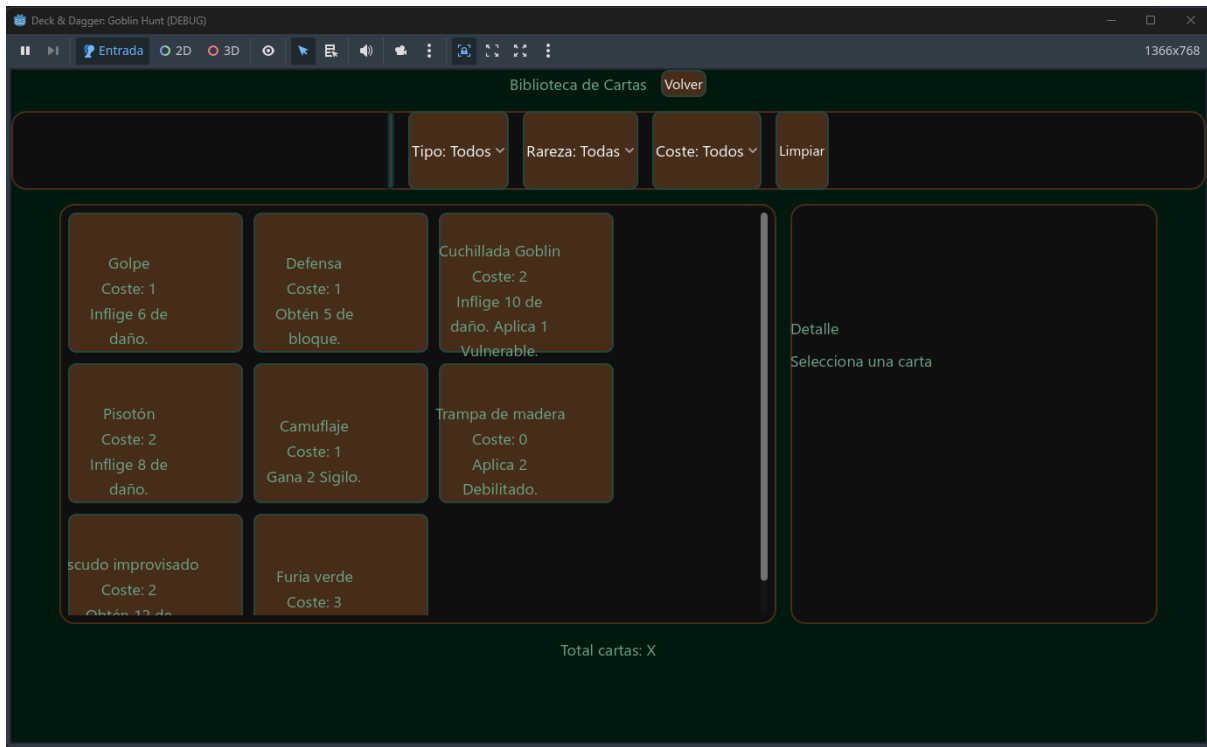
- Lista visual de cartas en venta
- Panel de detalle de la carta seleccionada
- Acciones de compra



2.7 Biblioteca de cartas

La biblioteca permite consultar todas las cartas disponibles en el juego. Incluye:

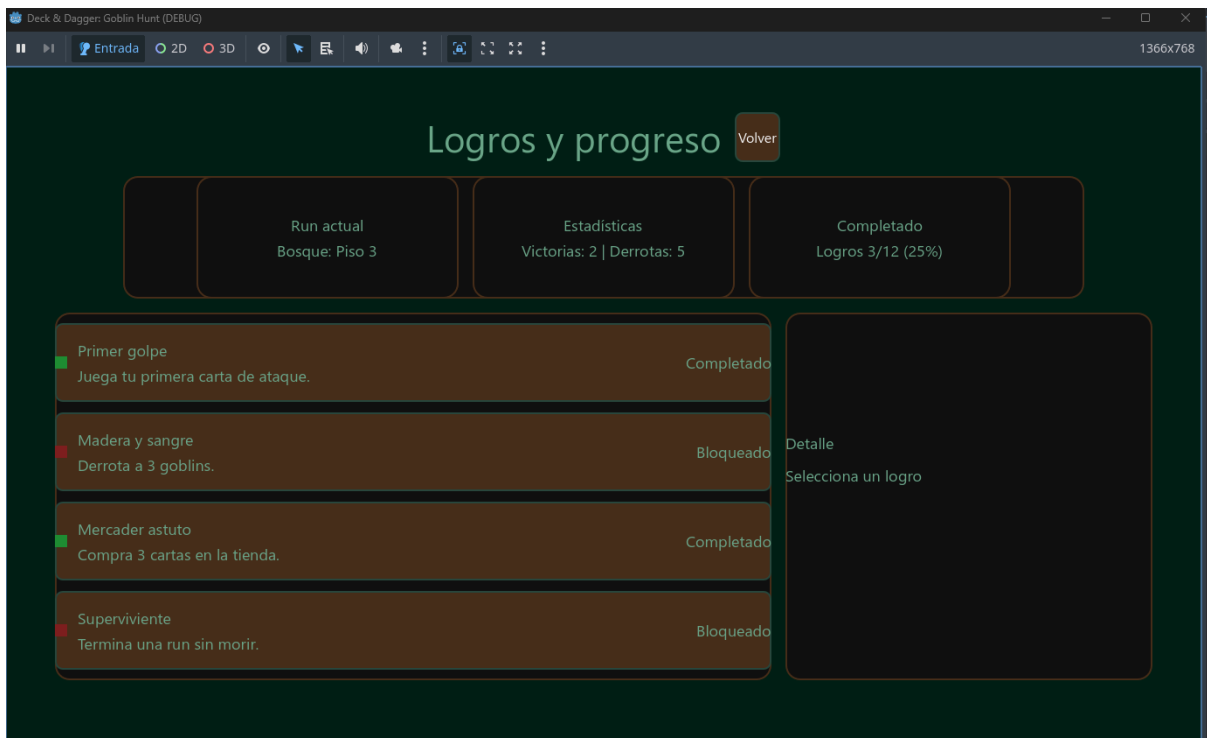
- Visualización en cuadrícula
- Filtros por tipo, coste y rareza
- Panel de detalle de la carta seleccionada



2.8 Logros y progreso

Esta pantalla muestra:

- Progreso general del jugador
- Lista de logros desbloqueados y bloqueados
- Detalle del logro seleccionado

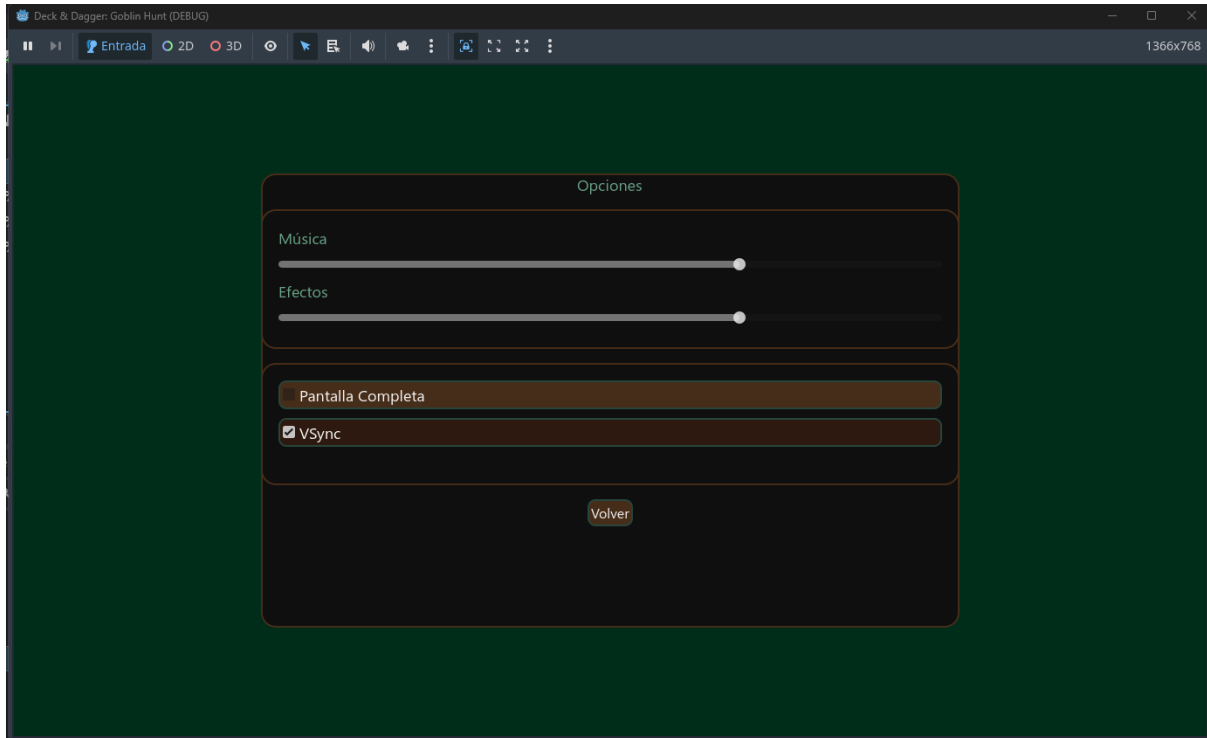


2.9 Opciones

La pantalla de opciones permite configurar:

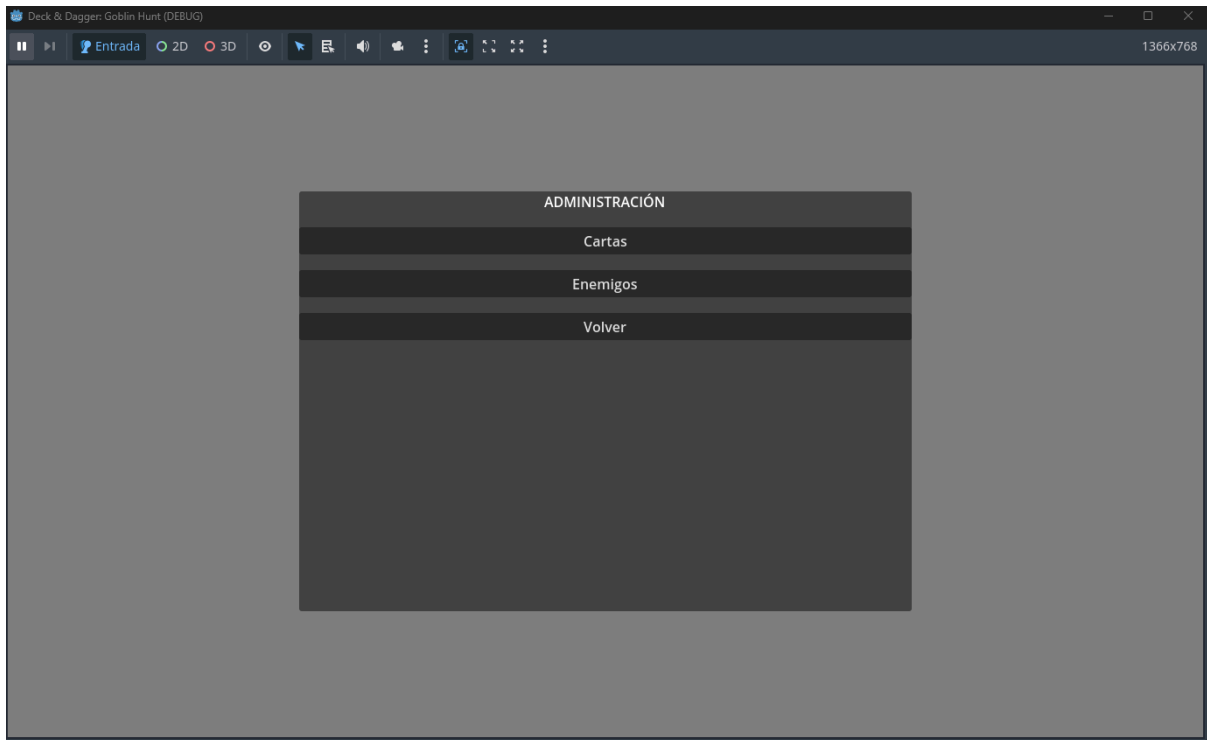
- Volumen de música
- Volumen de efectos
- Modo de pantalla (ventana / pantalla completa)
- VSync

Los cambios se aplican de forma automática.



2.10 Menu Admin

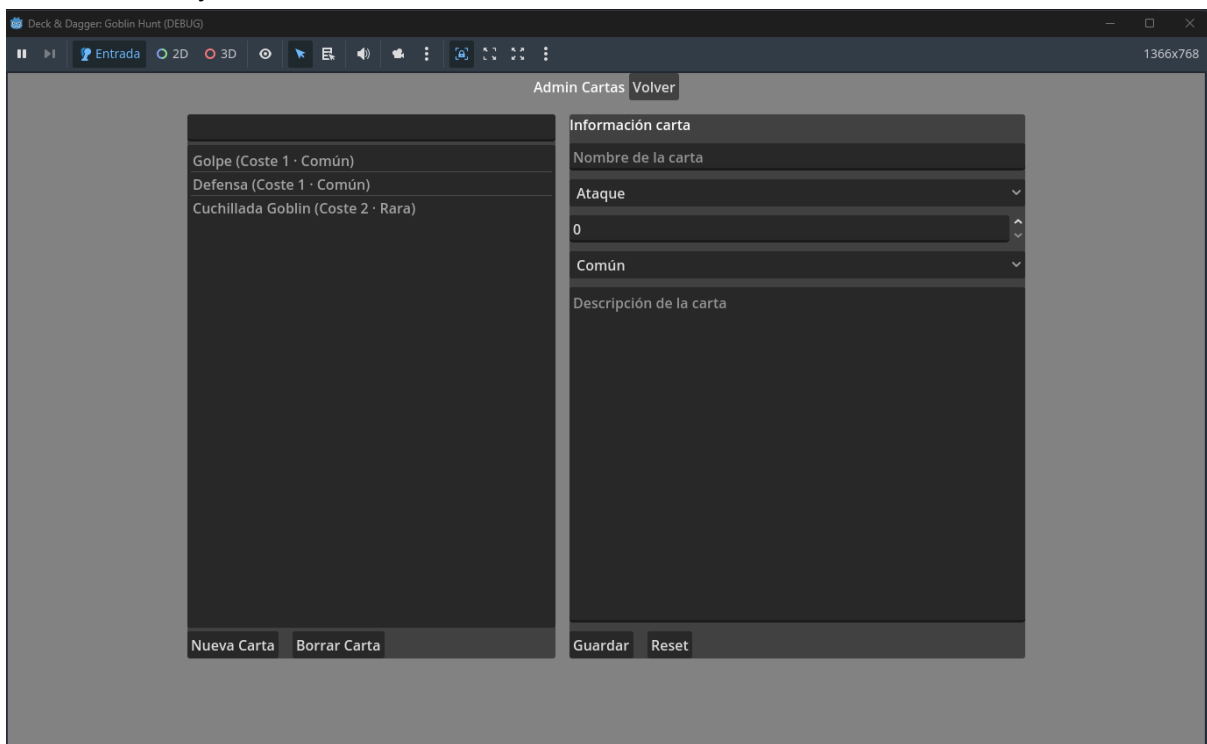
El menú de administración actúa como punto de acceso al back-office del sistema. Desde él se puede acceder a la gestión de los distintos elementos del juego.



2.11 Admin Cartas (CRUD)

La interfaz de administración de cartas permite realizar operaciones CRUD:

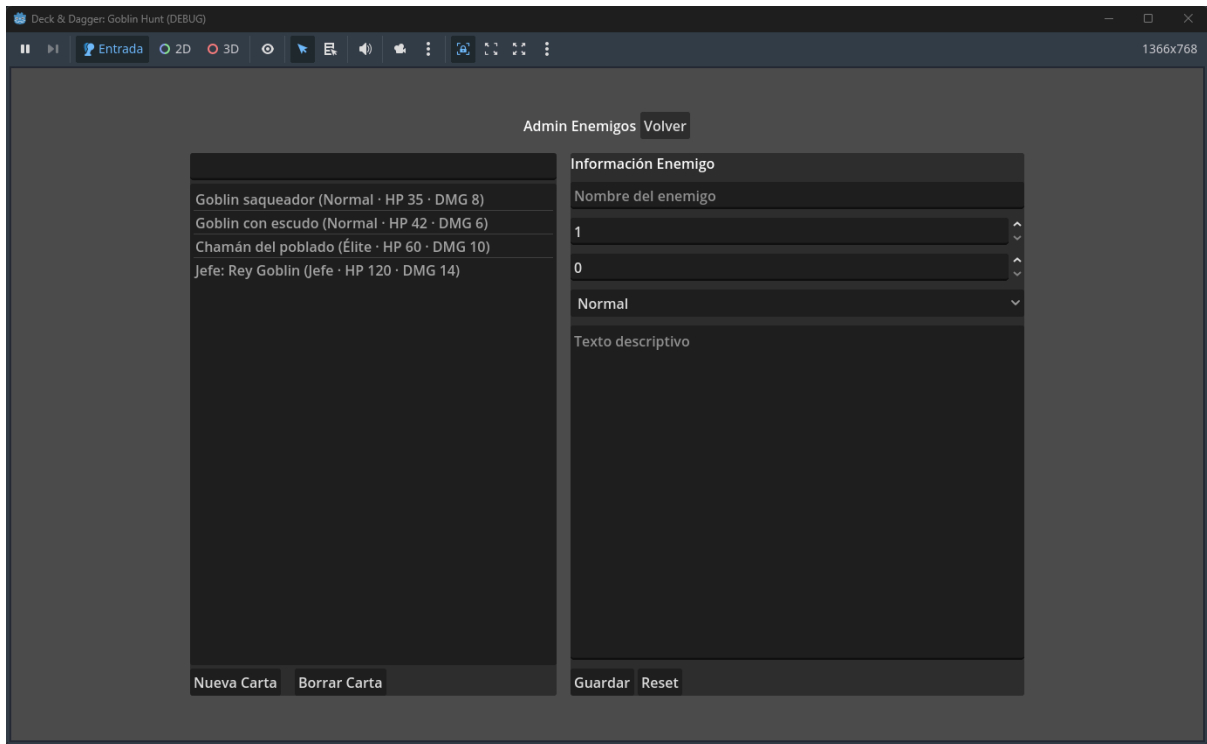
- Crear nuevas cartas
- Editar cartas existentes
- Eliminar cartas
- Buscar y filtrar cartas



2.12 Admin Enemigos (CRUD)

De forma similar a la gestión de cartas, esta interfaz permite administrar enemigos del juego, definiendo:

- Nombre
- Tipo
- Vida
- Daño
- Descripción



2.7. Plan de pruebas

Definición del alcance de la calidad

1.1. Objetivo del plan de pruebas

El objetivo de este Plan Maestro de Pruebas es definir qué aspectos críticos se controlarán para garantizar la calidad del software antes de su entrega final. En esta fase del proyecto no se ejecutan pruebas; se establecen los procedimientos, tipos de prueba, responsables, entornos y secuencia de ejecución que se aplicarán durante el desarrollo y validación del producto.

1.2. Alcance funcional

1.2.1. Gestión de usuario y progreso

Creación/selección de perfil de jugador.
Guardado y carga de progreso.
Persistencia de estado de partida (si aplica).

1.2.2. Sistema de cartas (Deckbuilder)

Carga de cartas desde base de datos.
Construcción de mazo, robo, descarte y mano.
Aplicación de efectos, costes y condiciones.

1.2.3. Sistema de combate

Turnos, acciones disponibles, resolución de efectos.
Cálculo de daño, bloqueos, estados alterados y sus duraciones.
Finalización de combate, recompensas y transiciones.

1.2.4. Sistema de enemigos

Carga de enemigos desde base de datos.
Patrones de ataque/IA básica, escalado (si existe), estados.
Integridad de estadísticas (vida, daño, habilidades).

1.2.5. Sistema de logros y desbloques

Desbloqueo de cartas por logros/nivel.
Registro de logros conseguidos.
Visualización en UI y persistencia.

1.2.6. Persistencia de datos (SQLite)

Integridad referencial del esquema.

Lecturas/escrituras de progreso, logros, cartas desbloqueadas.
Manejo de errores (DB ausente, corrupción, datos incompletos).

1.2.7. Interfaces de administración (CRUD)

Alta/edición/baja de cartas y enemigos (según alcance).
Validación de campos y consistencia de datos introducidos.

1.2.8. Interfaz de usuario (UI)

Navegación entre pantallas principales.
Consistencia visual y usabilidad en PC.
Mensajes de error y confirmaciones.

1.3. Alcance no funcional (RNF críticos)

1.3.1. Rendimiento (fluidez en PC Windows)

El combate debe mantenerse fluido y sin bloqueos perceptibles.
Cargas de pantallas y datos sin pausas prolongadas.

1.3.2. Estabilidad y robustez

El juego no debe cerrarse inesperadamente ante entradas no previstas.
El sistema debe gestionar estados inconsistentes (por ejemplo, datos incompletos en DB) con control de errores.

1.3.3. Persistencia fiable

El progreso, logros y desbloqueos deben mantenerse entre sesiones.
Las operaciones con SQLite deben ser atómicas y consistentes.

1.3.4. Usabilidad

Navegación clara, botones accesibles, feedback inmediato de acciones.
Evitar ambigüedad en combate (qué carta se juega, sobre qué objetivo, coste y efecto resultante).

1.3.5. Compatibilidad (Windows)

Correcto funcionamiento en ejecución exportada para Windows.
Rutas de ficheros y acceso a BD consistentes en el entorno objetivo.

1.3.6. Seguridad básica e integridad

Evitar corrupción o pérdida de progreso ante cierres inesperados.
Validación mínima de entradas en pantallas de administración.

2. Diseño de la estrategia de pruebas

2.1 Principios generales

La estrategia se basa en una ejecución progresiva, desde componentes aislados hasta validación integral del sistema. Se prioriza:

- Detectar fallos temprano (unitarias).
- Verificar integración entre subsistemas (integración/sistema).
- Confirmar cumplimiento de expectativas (aceptación).
- Controlar rendimiento y estabilidad (rendimiento/robustez).

2.2 Tipos de pruebas seleccionadas y justificación

2.2.1. Pruebas unitarias

Objetivo: validar la lógica de componentes aislados.

Aplicación en el proyecto:

- Cálculo de daño/defensa.
- Reglas de coste y jugada de cartas.
- Aplicación y expiración de estados.

Justificación: reduce defectos lógicos en mecánicas núcleo (combate y cartas).

2.2.2. Pruebas de integración

Objetivo: comprobar que subsistemas interactúan correctamente.

Aplicación en el proyecto:

- Cartas en combate (aplicación de efecto y actualización de UI).
- Enemigos en combate (turnos, decisiones, estados).
- Combate y progreso/logros (registro correcto tras finalizar).

Justificación: el riesgo principal del proyecto está en la interacción entre lógica de combate, datos y UI.

2.2.3. Pruebas de sistema

Objetivo: validar flujos completos end-to-end.

Aplicación en el proyecto:

- Inicio → selección de perfil → combate → recompensa → guardado → salida → carga.

Justificación: asegura que la aplicación cumple los casos de uso completos.

2.2.4. Pruebas de aceptación (usuario)

Objetivo: confirmar que el producto cumple expectativas funcionales y de usabilidad.

Aplicación en el proyecto:

- Pruebas guiadas con checklist (combate entendible, navegación clara, feedback visible).

Justificación: valida que el juego es utilizable y comprensible para un usuario final en PC.

2.2.5. Pruebas de rendimiento

Objetivo: asegurar fluidez y tiempos de respuesta aceptables.

Aplicación en el proyecto:

- Combate prolongado con múltiples estados y cartas.
- Carga de biblioteca de cartas / pantallas con listas.

Justificación: un deckbuilder debe mantener respuesta inmediata para ser jugable.

2.2.6. Pruebas de seguridad básica / integridad

Objetivo: garantizar consistencia de datos y evitar pérdidas.

Aplicación en el proyecto:

- Guardados consecutivos, cierre forzado, reanudación.
- Validación de entradas en administración (evitar datos imposibles).

Justificación: la persistencia (SQLite) es parte crítica del valor del producto.

2.3 Secuencia de ejecución propuesta

La ejecución seguirá el siguiente orden lógico:

1. Unitarias: validar reglas internas (cartas/daño/estados).
2. Integración: validar cooperación entre módulos (cartas, combate, enemigos, DB).
3. Sistema: validar flujos completos de usuario (end-to-end).
4. Rendimiento: medir fluidez en escenarios de carga real.
5. Aceptación: validar experiencia final y requisitos de usabilidad.
6. Regresión (continua): repetir subconjuntos de pruebas tras correcciones o cambios relevantes.

2.4 Entornos de prueba

Se definen los siguientes entornos:

Entorno de desarrollo

- Godot Editor (ejecución local).
- Base de datos SQLite de desarrollo.

Entorno de pruebas

- Ejecutable exportado para Windows (build de prueba).
- Base de datos SQLite de pruebas (copia controlada con datos semilla).
- Objetivo: validar comportamiento real fuera del editor.

Datos de prueba

- Conjunto mínimo: cartas/enemigos básicos para comprobar flujos.
- Conjunto ampliado: múltiples cartas y enemigos para estrés funcional y rendimiento.

2.5 Roles y responsabilidades

Dado que el proyecto se desarrolla de forma individual, se definen los roles:

Responsable de pruebas técnicas (Desarrollador)

- Diseña y ejecuta pruebas unitarias/integración/sistema/rendimiento.
- Registra incidencias y verifica correcciones (regresión).

Responsable de validación de aceptación (Usuario final simulado)

- Ejecuta checklist de aceptación orientado a jugabilidad y comprensión.
- Reporta problemas de usabilidad, claridad de UI y coherencia.

3 Gestión de proyecto

3.1. Scrum

Introducción

El desarrollo del proyecto Deck & Dagger – Goblin Hunt se ha gestionado parcialmente mediante la metodología ágil Scrum, dado que se trata de un proyecto software con un alto componente iterativo, múltiples funcionalidades interrelacionadas y necesidad de validación progresiva del producto.

Scrum permite dividir el desarrollo en sprints de duración fija, facilitando el control del progreso, la adaptación a cambios y la trazabilidad entre requisitos funcionales, tareas técnicas y resultados obtenidos. Esta metodología resulta especialmente adecuada para proyectos de videojuegos, donde la jugabilidad, el equilibrio y la integración de sistemas requieren ajustes continuos.

Durante el desarrollo del proyecto, parte de la implementación técnica se realizó de forma paralela a la elaboración de la documentación académica. Por este motivo, se ha aplicado una planificación Scrum retrospectiva para los sprints ya ejecutados. Los sprints S11 y S12 son los de desarrollo del videojuego, en este caso son los incluidos en el KANBAN.

2. Kanban

Para la gestión de las tareas y el seguimiento del proyecto se ha utilizado un tablero Kanban en GitHub Projects, asociado al repositorio del proyecto `deck---dagger--goblin-hunt`.

El tablero Kanban se estructura en las siguientes columnas:

Backlog
Sprint Backlog
In Progress
Testing
Done

Cada tarea incluye información relativa al sprint, fase de desarrollo.

Los Sprints S11 y S12 son los derivados del desarrollo que aunque ya se han realizado múltiples tareas previamente se han añadido esas tareas a estos dos Sprints.

El profesor ha sido invitado al tablero Kanban para permitir el seguimiento del proyecto y la verificación del uso de la metodología Scrum.

[SCRUM · @SesarDev's Deck & Dagger](#)

3. Listado y planificación de Sprints

Sprint S1

Fechas: 01/10/2025 – 15/10/2025

Objetivo: Definir la idea del proyecto

Tareas:

- Propuesta de proyecto
- Elección del tipo de aplicación
- Selección de tecnologías principales

Tipo: Documentación

Fase: Análisis

Sprint S2

Fechas: 03/11/2025 – 09/11/2025

Objetivo: Análisis del contexto empresarial

Tareas:

- Análisis del sector
- Estructura de la empresa tipo

Tipo: Documentación

Fase: Análisis

Sprint S3

Fechas: 10/11/2025 – 16/11/2025

Objetivo: Estudio del mercado y viabilidad inicial

Tareas:

- Estudio de mercado
- Análisis DAFO

Tipo: Documentación

Fase: Análisis

Sprint S4

Fechas: 17/11/2025 – 23/11/2025

Objetivo: Innovación y complejidad del proyecto

Tareas:

- Innovación, dificultad e investigación

Tipo: Documentación

Fase: Análisis

Sprint S5

Fechas: 24/11/2025 – 30/11/2025

Objetivo: Definir los requisitos del sistema

Tareas:

- Especificación de requisitos funcionales y no funcionales

Tipo: Documentación

Fase: Análisis

Sprint S6

Fechas: 01/12/2025 – 07/12/2025

Objetivo: Modelado funcional del sistema y diseño visual inicial

Tareas:

- Diagrama de casos de uso
- Casos de uso críticos
- Bocetos de interfaces

Tipo: Documentación

Fase: Diseño

Sprint S7

Fechas: 08/12/2025 – 14/12/2025

Objetivo: Definir la viabilidad del proyecto y establecer una planificación temporal realista

Tareas:

- Estudio viabilidad
- Ciclo de vida, Gantt

Tipo: Documentación

Fase: Diseño

Sprint S8

Fechas: 15/12/2025 – 21/12/2025

Objetivo: Diseñar la arquitectura lógica, estructural y visual completa del sistema

Tareas:

- Diagrama Entidad-Relación
- Esquema relacional
- Diagrama de clases
- Diseño de interfaces final

Tipo: Documentación

Fase: Diseño

Sprint S9

Fechas: 22/12/2025 – 28/12/2025

Objetivo: Planificar la estrategia de pruebas para garantizar la calidad del sistema

Tareas:

- Plan de pruebas

Tipo: Documentación

Fase: Planificación

Sprint S10

Fechas: 29/12/2025 – 04/01/2026

Objetivo: Planificar y estructurar el desarrollo del proyecto mediante la metodología Scrum

Tareas:

- Definición de los sprints del proyecto
- Creación del tablero Kanban
- Asignación de tareas a sprints
- Relación de tareas con requisitos funcionales y no funcionales

Tipo: Desarrollo

Fase: Desarrollo

Sprint S11

Fechas: 05/01/2026 – 11/01/2026

Objetivo: Construir la infraestructura completa del juego, incluyendo pantallas, navegación, base de datos SQLite, sistema de cartas y definición de enemigos.

Tareas:

A. Estructura del proyecto y navegación

- Crear escena base del juego
- Crear gestor de escenas (SceneManager)
- Implementar navegación entre pantallas
- Definir flujo menú → partida → retorno
- Configurar autoloads necesarios

B. Pantallas / Ventanas

- Crear pantalla de menú principal
- Crear pantalla de nueva partida
- Crear pantalla de continuar partida
- Crear pantalla de perfil del jugador
- Crear pantalla de biblioteca de cartas
- Crear pantalla de combate (estructura)
- Crear pantalla de mapa de nodos (estructura)
- Crear pantalla admin (CRUD)
- Conectar todas las pantallas mediante navegación

C. Base de datos SQLite (Godot)

- Integrar SQLite en el proyecto Godot
- Diseñar esquema de base de datos
- Crear tabla jugador
- Crear tabla progreso
- Crear tabla cartas
- Crear tabla enemigos
- Crear tabla logros
- Implementar conexión y cierre de BD

- Implementar guardado automático de progreso
- Implementar carga automática al iniciar partida
- Validar integridad de datos

D. Sistema de cartas

- Definir modelo de carta (ataque, defensa, habilidad)
- Implementar carga de cartas desde BD
- Implementar mazo del jugador
- Implementar mano de cartas
- Implementar descarte
- Implementar uso de cartas
- Integrar cartas con la interfaz
- Control de cartas desbloqueadas

E. Enemigos

- Definir modelo de enemigo
- Definir enemigos normales
- Definir enemigos jefe
- Implementar carga de enemigos desde BD
- Integrar enemigos con combate (estructura)

F. Validación

- Pruebas funcionales de navegación
- Pruebas de persistencia de datos

Tipo: Desarrollo

Fase: Desarrollo

Sprint S12

Fechas: 12/01/2026 – 18/01/2026

Objetivo: Completar la jugabilidad del juego mediante el sistema de combate, tienda, recompensas y logros, dejando el proyecto completamente funcional.

Tareas:

A. Sistema de combate

- Implementar combate 1 vs 1
- Implementar sistema de turnos simple
- Implementar energía por turno
- Implementar robo de cartas por turno
- Implementar uso de cartas en combate
- Implementar daño y defensa
- Implementar condiciones de victoria
- Implementar condiciones de derrota
- Implementar combate contra jefes
- Mostrar intenciones básicas del enemigo

B. Recompensas post-combate

- Implementar recompensa de oro
- Implementar recompensa de experiencia
- Implementar recompensa de cartas
- Implementar elección entre recompensas
- Guardar recompensas en progreso

C. Tienda

- Crear pantalla de tienda
- Generar cartas aleatorias para tienda
- Implementar compra de cartas
- Descontar oro al comprar
- Añadir carta comprada al progreso
- Guardar compras en BD

D. Sistema de logros

- Definir logros del juego
- Implementar desbloqueo de logros
- Guardar logros en BD
- Mostrar logros en perfil
- Mostrar notificación de logro desbloqueado

E. Integración y pulido

- Integrar combate con mapa de nodos
- Integrar tienda y recompensas en el flujo
- Ajustes de balance inicial
- Ajustes de interfaz y experiencia de usuario
- Refactor de código final

F. Pruebas finales

- Pruebas funcionales del combate
- Pruebas de recompensas y tienda
- Pruebas de logros
- Pruebas generales del sistema

Tipo: Desarrollo

Fase: Desarrollo

Sprint S13

Fechas: 19/01/2026 – 25/01/2026

Objetivo: Cierre del proyecto y preparación de la defensa

Tareas:

- Manual técnico
- Presentación PowerPoint

- Preparación de la defensa

Tipo: Documentación / Pruebas

Fase: Pruebas / Documentación

4. Conclusión

Aunque no se ha realizado parte del videojuego utilizando metodología ágil si se ha puesto en marcha el repositorio de Github desde el comienzo ya que se ha ido creando contenido del proyecto desde dos ordenadores diferentes compartiendo con Github Desktop el código.